Deep Dive in the Poisson Mode Registers, Wells and Semantic Templates

Compiled by D. Gueorguiev, 10/23/2026

Table of Contents

The Poisson-Mode Registers and Wells ***3***

Notation and Introductory Notes **3**

Architecture Hyperparameters 3

State Variables 3

Learnable Parameters of the Poisson Modes 3

Derived Quantities 4

Diagnostic Quantities 4

Disambiguation: Registers, Modes and Wells 5

Principle of Operation **5**

From Slot Registers to Poisson Modes 5

The Occupation Is a Poisson Mean 6

The Wells and Their Force 6

Two Ways to Integrate the Wells: PM1 and PMX 6

First Order: FO-b on the Well Base 6

The Code: model_parf_multixi.py **6**

Configuration Fields 6

Construction of the Parameters 7

The Occupation: poisson_mode_occupation 8

The Force: poisson_mode_force 9

Where the Force Enters: _layer_forces 9

The Wells' Quadratic: poisson_mode_quadratic 10

The PMX Layer Step 10

The Exact Substeps in cfc_baoab.py 11

The Notebook: Switches, Tags and Guards 13

What the Trained Wells Do **15**

Position, Not Content 15

A Fixed Landscape Beyond the First Tokens 16

Sixteen Modes Against Sixty-Four 16

The Diagnostic Tools 16

Toward Semantic Templates ***17***

What a Template Needs **17**

Anatomy: Three Kinds of Parameter 17

Tiers and Stages 18

Modes as Matchers: Form Without Function **18**

The Narrow-Well Deliberations and Experiments **18**

Why the Trained Modes Cannot Match 18

Low-Rank Matching 19

Implementation: poisson_match_rank 19

Verification 20

The Short Training Run 21

The Half-Life Range 21

The Particle Store: Making Allocation Real (Stage 0") **22**

Why Neither Registers nor Modes Allocate 22

The Design: a Write-Once, Block-Causal Store 22

Decay Decides the Class 23

Implementation: particle_store 24
The Notebook: Switches, Tag and Guard 27
Verification of the Store 28
A Pre-Existing Bug in the Stage 0' Matcher Under Checkpointing 28
What the First Build Does Not Yet Do 29
The 2B-Token Run 29
Why 2B, and What It Must Decide 29
The Schedule and the Snapshots 29
Snapshot Code (Cell 6) 29
Reading a Snapshot: ckpt= and the Harness 30
Per-Snapshot Diagnostics 30
The Planned Template Pair 31
Pre-Registered Calls 31
Paired Evaluation: the Shared Random Stream 32
References 32
Appendix 33
A1. The Poisson Thinning Argument 33
A2. The Exact Damped Flow of an Indefinite Spring 33
A3. The Overdamped Relaxation 34
A4. Concentration of Distances and Low-Rank Matching 34
A5. How an Occupation Averages Content Away 34
A6. The k-Means Initialisation and the Clustered Initial Cloud 35
A7. The Fixed Landscape: What the Static Test Shows and What It Does Not 35
A8. Paired Evaluation and the Seed 35

The Poisson-Mode Registers and Wells

Notation and Introductory Notes

This deep dive explains the Poisson-mode registers and the wells they create, as implemented in `model_parf_multixi.py` [1] and the ladder notebook [2], and the work that prepares them to serve as the matchers of semantic templates. It follows the notation of the book's §9 (paragraph *Poisson-mode registers: the bosonic v2, trained*, equations pm-occupation and pm-potential) [3] and of the SPLM deep dive [4]. Where the book writes the layer index as l, this document writes ℓ, as the SPLM deep dive does.

Architecture Hyperparameters

d – hidden width; $d = 384$ in every Gen 3 arm discussed here. In code `cfg.d`.

L – number of integration layers (trained layer steps); $L = 2$ in the 0.5B- and 2B-token arms. In code `cfg.L`; in the notebook `LADDER_L`.

Δt – the trained layer step; $\Delta t = 4$, so the stack integrates over $L\,\Delta t = 8$. In code `cfg.dt`.

K – number of Poisson modes, shared by the layers in the default design. In the notebook `POISSON_MODES` (0 = off; tag `pm<K>`); in code `cfg.poisson_modes`. The arms discussed use $K \in \{16{,}64\}$; the dose-response adds $K = 2$.

M – the number of a token's strongest wells whose rank part joins the exact flow under PMX. In the notebook `POISSON_WELLS_EXACT_MODES` (default 16); in code `cfg.poisson_wells_exact_modes`.

$[\tau_{\min}, \tau_{\max}]$ – the range, in tokens, over which the modes' initial half-lives are log-spaced. In the notebook `POISSON_HALFLIFE_MIN`, `POISSON_HALFLIFE_MAX` (defaults 4 and 128; tag `pmhl<min>t<max>` otherwise); in code `cfg.poisson_halflife_min`, `cfg.poisson_halflife_max`.

r – the rank of the content-selective matcher (Stage 0'; see *The Narrow-Well Deliberations and Experiments*). In the notebook `POISSON_MATCH_RANK` (0 = off; tag `pmlr<r>`); in code `cfg.poisson_match_rank`.

γ – the scalar damping of the second-order step; $\gamma = 0.1$. In code `cfg.fixed_gamma`.

c_{cap} – an optional smooth bound on the well depths. In the notebook `POISSON_DEPTH_CAP` (None = off; tag `pmcap<cap>`).

State Variables

x – input token-id tensor; in code `x`. Shape: (B, T).

$h_t^{(\ell)}$ – the state of token t at the input of layer ℓ, a point of the semantic space $\mathbb{R}^d$. In code `h` or `h_in`. Shape: (B, T, d). At $\ell = 0$ it is the embedding of the token (not LayerNorm-scaled; see *Why the Trained Modes Cannot Match*); every later layer receives a LayerNorm-projected state of norm about $\sqrt{d} \approx 19.6$.

$v_t^{(\ell)}$ – the velocity carried between layers by the second-order step; in code `v`. A first-order (FO-b) model has none.

$\mathfrak{m}_t$ – the token's semantic mass; in code `m_b`. Shape $(B, T, 1)$.

ξ_t – the context channels: five normalised causal exponential averages of earlier states, with trained decay coefficients near $(0.33, 0.52, 0.89, 0.99, 0.99)$; in code `xis`. They drive the wells of V_θ, the core potential, not the Poisson-mode wells.

Learnable Parameters of the Poisson Modes

$\mu_v \in \mathbb{R}^d$ – the centre of mode v. In code `pm_mu`, shape (K, d), initialised from $\mathcal{N}(0, I)$, so $\|\mu_v\| \approx \sqrt{d}$. Under the content-selective matcher the centres are per layer, $\mu_{\ell,v}$, `pm_mu` of shape (L, K, d).

κ_v^2 – the inverse squared width of mode v, stored as its logarithm. In code `pm_log_kappa2`, shape $(K,)$, initialised at $\ln \kappa_v^2 = -\ln d$, so the exponent of the overlap starts as $\|h - \mu_v\|^2 / d$.

$\lambda_v \in (0,1)$ – the per-token survival probability of a particle in mode v, stored as a logit. In code `pm_logit_lambda`, shape $(K,)$. Its half-life is

$$\tau_v = \frac{\ln 2}{-\ln \lambda_v}, \qquad \lambda_v = 2^{-1/\tau_v} \qquad \text{(pm.1)}$$

and the initial half-lives are log-spaced over $[\tau_{\min}, \tau_{\max}]$.

$\varepsilon_{\ell,v}$ – the depth of mode v's well at layer ℓ, of either sign: positive is attractive, negative repulsive. In code `pm_depth`, shape (L, K), initialised at zero, so the wells exert no force at step 0. With a cap the depth the force uses is $a_{\ell,v} = c_{\text{cap}} \tanh(\varepsilon_{\ell,v}/c_{\text{cap}})$ (`pm_effective_depth`); without one $a_{\ell,v} = \varepsilon_{\ell,v}$.

Content-selective matcher only (Stage 0'): $P_\ell \in \mathbb{R}^{r \times d}$, a learned projection per layer (`pm_P`, shape (L, r, d)); $c_{\ell,v} > 0$, a learned width factor (`pm_log_c`, its logarithm, shape (L, K)); and two buffers, $\bar{D}^2_\ell$, a running median of the squared projected distance between tokens (`pm_D2`), and a flag that records whether layer ℓ's matcher has been initialised from data (`pm_match_ready`).

Derived Quantities

$E_v(s)$ – the overlap of token s with mode v, and the rate at which token s creates particles in mode v (the single-particle overlap identity of [5]):

$$E_v(s) = \exp(-\kappa_v^2 \|h_s - \mu_v\|^2) \qquad \text{(pm.2)}$$

In code `E`, shape (B, T, K).

$\bar{n}_v(t)$ – the mean occupation of mode v that token t sees, built from the strict past:

$$\bar{n}_v(t) = \textstyle\sum_{s<t} \lambda_v^{t-1-s} E_v(s), \qquad \bar{n}_v(t+1) = \lambda_v \bar{n}_v(t) + E_v(t), \qquad \bar{n}_v(1) = 0 \qquad \text{(pm.3)}$$

In code `phi`, shape (B, T, K). The book calls it the occupation; the code calls it `phi` for historical reasons.

$V_{\text{pm}}(h; t)$ – the potential of the Poisson-mode wells felt by token t at layer ℓ:

$$V_{\text{pm}}(h; t) = -\textstyle\sum_{v=1}^{K} \bar{n}_v(t)\, a_{\ell,v}\, e^{-\kappa_v^2 \|h-\mu_v\|^2} \qquad \text{(pm.4)}$$

$w_v(t)$ – the weight of well v in the force, and the token's force:

$$w_v(t) = 2\kappa_v^2\, a_{\ell,v}\, \bar{n}_v(t)\, E_v(t), \qquad f_{\text{pm}}(h_t) = -\nabla_h V_{\text{pm}} = -\textstyle\sum_v w_v(t)\,(h_t - \mu_v) \qquad \text{(pm.5)}$$

In code `w` and `f`. The force is a pull of strength w_v toward each centre (a push where $w_v < 0$).

α_t, r_v, $\mathrm{d}W_v$ – the pieces of the wells' Hessian used by PMX: $\alpha_t = \sum_v w_v(t)$ (the isotropic curvature, `alpha`), $r_v = h_t - \mu_v$ (`r`) and $\mathrm{d}W_v = 2\kappa_v^2 w_v(t)$ (`dW`):

$$\nabla_h^2 V_{\text{pm}} = \alpha_t I - \textstyle\sum_v \mathrm{d}W_v\, r_v r_v^{\mathsf{T}} \qquad \text{(pm.6)}$$

$x_v^* = 1/(\sqrt{2}\,\kappa_v)$ – the inflection radius of well v, where its pull is strongest.

Diagnostic Quantities

p_v – mode v's share of the wells' force over the evaluated tokens, $p_v = \sum_t \|w_v(t)(h_t - \mu_v)\| \,/\, \sum_t \sum_u \|w_u(t)(h_t - \mu_u)\|$.

N_{eff} – the effective number of modes in use:

$$N_{\text{eff}} = \exp(-\textstyle\sum_v p_v \ln p_v) \qquad \text{(pm.7)}$$

It equals K when every mode carries the same share and 1 when one mode carries all of it.

Selectivity – the median, over modes, of the coefficient of variation of the overlap across tokens, $\mathrm{std}_s E_v(s) / \mathrm{mean}_s E_v(s)$. Trained isotropic modes read 0.01–0.04.

Position share R^2_{pos} – the share of the occupations' variance explained by position alone, each $\bar{n}_v(t)$ replaced by its mean over the *other* sequences at the same position.

Content swap – the perplexity change when every sequence is given another sequence's occupations (same positions, different content).

Disambiguation: Registers, Modes and Wells

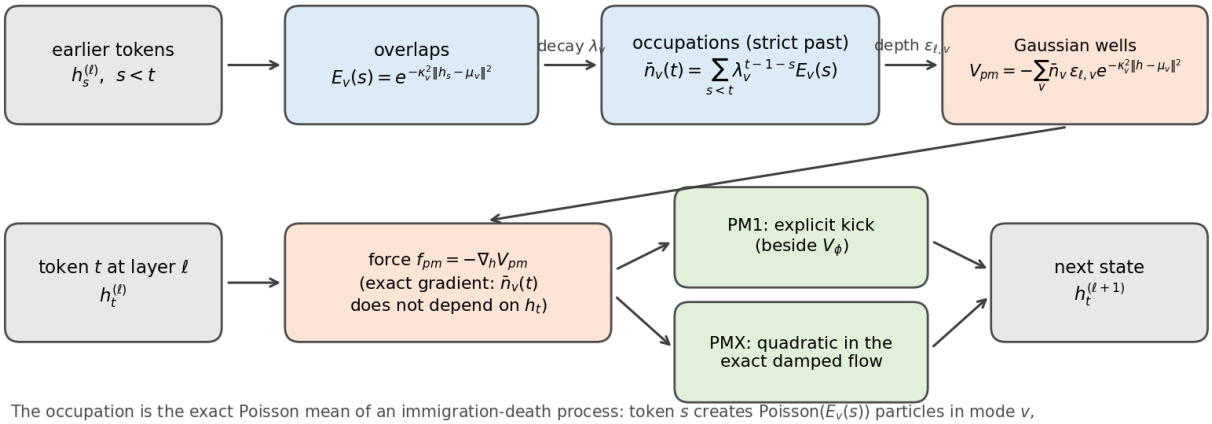
Three words are overloaded in this code base, and the deep dive keeps them apart.

Register. The Fock models have **slot registers**: M always-occupied slots whose content is renewed layer by layer and which reach the tokens through the reverse channel. The **Poisson-mode registers** are a different mechanism: K modes whose occupations are Poisson means. In this document "register" without a qualifier means the Poisson-mode register.

Mode. The integrator speaks of the **stiff modes** of V_θ: the leading eigendirections of its curvature, which the exact flow integrates (`lr_U`, `lr_kappa`). A **Poisson mode** is a Gaussian cloud in $\mathbb{R}^d$ with an occupation. The two meet only in PMX, where a token's strongest Poisson wells join the stiff modes in one eigensolve.

Well. V_θ has its own wells: eight joint Gaussian attractors whose centres and precisions are read from the context channels ξ. The **Poisson-mode wells** are the K Gaussian terms of (pm.4), whose depths are occupations. In this document "the wells" means the Poisson-mode wells.

Principle of Operation



The occupation is the exact Poisson mean of an immigration-death process: token s creates Poisson($E_v(s)$) particles in mode v, each survives a token with probability λ_v. Linear in $\bar{n}$, so carrying the mean is exact. Schematic; no data.

The Poisson-mode wells in one layer step.

From Slot Registers to Poisson Modes

Book Remark rem:dp-registers measured what the trained slot registers implement: their active fraction is 0.9995–1.000 at every layer, so their particle number never changes, and their salience is not an intensity. The Poisson-mode registers were built to make both hold by construction. Each token creates particles in each mode at a rate equal to its overlap with the mode, and each particle survives from one token to the next with probability λ_v. That is an immigration–death process, one per mode, and the token at t feels Gaussian wells whose depths are the current occupations.

The Occupation Is a Poisson Mean

Book Proposition prop:pm-poisson states that if mode v receives at token s a Poisson-distributed number of particles with mean $E_v(s)$, and every particle survives each token independently with probability λ_v, then the number of particles at token t is Poisson with mean $\bar{n}_v(t)$ of (pm.3), independently across modes. The proof is a binomial thinning argument (Appendix A1). The consequence for the code is that carrying the mean is exact: the potential (pm.4) is linear in the occupation, so the expected force over the particle process equals the force of the mean, and no sampling is needed.

The occupation uses the strict past ($s < t$). It therefore does not depend on h_t, and the force (pm.5) is the exact gradient in h_t of a potential: the wells keep the model conservative. The loss still reaches earlier tokens through $\bar{n}_v(t)$ (the *live* gradient convention of Gen 3).

The Wells and Their Force

Each well is a Gaussian with centre μ_v and width $1/\kappa_v$, scaled by the occupation and the depth. Its pull on a token at distance ρ is proportional to $\rho\, e^{-\kappa_v^2 \rho^2}$, zero at the centre, largest at the inflection radius x_v^*, and negligible a few widths out. A positive depth attracts; a negative depth repels. Earlier context deepens the wells that later tokens fall into, and the half-life sets how long a deposit lasts.

Two Ways to Integrate the Wells: PM1 and PMX

PM1 adds f_{pm} to the conservative force beside the pair force f_ϕ, as a plain kick: the integrator treats it like any other force evaluated at the step's midpoint.

PMX integrates the wells exactly. At the start of each layer step it freezes the wells' quadratic, (pm.6) at $h_0 = h_t$ with the occupations fixed, and joins it to V_θ's retained low-rank modes in one frozen spring on all of $\mathbb{R}^d$:

$$K_{\text{spring}} = \alpha_t I + U\,\text{diag}(\kappa)\,U^\top, \qquad \mathfrak{m}\,\ddot{x} = f - K_{\text{spring}}(x - h_0) - \gamma\,\mathfrak{m}\,\dot{x} \qquad \text{(pm.8)}$$

The columns of U span V_θ's retained stiff modes and the rank part of the token's M strongest wells; the modes in that span have stiffness $\kappa + \alpha_t$ and the complement α_t, either sign. The flow is solved in closed form for every damping regime (Appendix A2), so damping is exact on the whole space and no separate friction (O) step is applied. What the frozen quadratic misses, the remainder of the wells' force and V_θ's soft modes, stays in the kick.

First Order: FO-b on the Well Base

The first-order twin FO-b replaces (pm.8) by the gradient flow with the same frozen spring, $\dot{x} = \mu_{\text{FO}}\,(f - K_{\text{spring}}(x - h_0))$ with mobility $\mu_{\text{FO}} = \Delta t/(2\mathfrak{m})$, solved exactly per direction (Appendix A3), and carries no velocity between layers.

The Code: model_parf_multixi.py

Configuration Fields

The configuration class documents the mechanism where it declares it. The comment on `poisson_modes` states the process and why the mean is exact:

```
# PM1 (protocol SS5.15, 2026-10-05): bosonic Poisson-mode registers. K
# shared mode prototypes mu_v; each token creates particles in mode v at
# rate equal to its overlap exp(-kappa_v^2 |h_s - mu_v|^2), and each
# particle survives to the next token with probability lambda_v. The
# occupation phi_v(t) = sum_{s<t} lambda_v^(t-1-s) overlap_v(s) is then
# EXACTLY the Poisson mean of an immigration-death process. The force is
# -grad_h of U = -sum_v phi_v(t) a_v exp(-kappa_v^2 |h - mu_v|^2): linear
# in phi, so carrying only the mean is exact. Strict past (s < t), so phi
# does not depend on h_t and the force is a gradient in h_t. 0 = off.
poisson_modes: int = 0
```

The half-life range and the PMX switches:

```
poisson_halflife_min: float = 4.0     # tokens; per-mode half-lives are
poisson_halflife_max: float = 128.0   # log-spaced over this range

# PMX (protocol SS5.15, 2026-10-10): integrate the Poisson-mode wells
# exactly. Their quadratic at the layer step's start (phi fixed) -- an
# isotropic curvature alpha plus an indefinite rank-K part -- joins V_theta's
# low-rank part in the exact damped flow, which then covers all of R^d
# (no O-step); the kick keeps only what that misses (13% of PM1's layer-1
# wells force with the rank part of the top 16 wells, 4.7% with all of
# them; debug/pm_wells_curvature.py). Vtheta is
# treated exactly as under SR2: its retained low-rank modes go into the
# exact flow, the soft ones (lowrank_max_modes) stay in the kick. Needs
# poisson_modes > 0, lowrank_damped_flow and langevin_T = 0. False = off
# (bit-identical).
poisson_wells_exact: bool = False
# PMX's rank part per token: every well's force and isotropic curvature go
# into the exact flow, but only the rank part W of the token's M strongest
# wells (by force) joins the eigensolve; the rest of W stays in the kick
# (13% of PM1's layer-1 wells force at M = 16). lowrank_max_modes + M must
# be <= 32: the GPU's batched solvers stop at 32 x 32 (an 80 x 80 version
# ran at > 26 s/step, 2026-10-10).
poisson_wells_exact_modes: int = 16
```

Construction of the Parameters

The parameters are created only when the modes are switched on, so the default model draws the same random stream and keeps the same `state_dict` (every existing arm stays bit-identical). The depths start at zero, so the force is exactly zero at step 0 and the depths receive a gradient from step 1:

```
# PM1 Poisson-mode registers. Created ONLY when enabled, so the
# default model draws the same RNG stream and keeps the same
# state_dict. Well depths start at zero: the force is exactly zero
# at step 0 and the depths receive a gradient from step 1.
self.pm_mu = None
_K = int(getattr(cfg, "poisson_modes", 0) or 0)
_cap = getattr(cfg, "poisson_depth_cap", None)
if _cap is not None and (_K <= 0 or not _cap > 0):
    raise ValueError("poisson_depth_cap needs poisson_modes > 0 and a positive cap, "
                     f"got poisson_modes={_K}, poisson_depth_cap={_cap}")
_R = int(getattr(cfg, "poisson_match_rank", 0) or 0)
if _R > 0:
    if _K <= 0:
        raise ValueError("poisson_match_rank needs poisson_modes > 0")
    if getattr(cfg, "poisson_wells_exact", False):
        raise ValueError("poisson_match_rank: the low-rank matcher runs on the explicit kick only "
                         "(poisson_wells_exact = False)")
    if _R > cfg.d:
        raise ValueError(f"poisson_match_rank = {_R} > d = {cfg.d}")
if _K > 0 and _R > 0:
    # Content-selective modes: set from the token states on the first training forward
    # (_pm_match), so no RNG is drawn here.
    self.pm_mu = nn.Parameter(torch.zeros(cfg.L, _K, cfg.d))
    self.pm_P = nn.Parameter(torch.zeros(cfg.L, _R, cfg.d))
    self.pm_log_c = nn.Parameter(
        torch.full((cfg.L, _K), math.log(float(getattr(cfg, "poisson_match_c_init", 1.0)))))
    self.register_buffer("pm_D2", torch.ones(cfg.L))
    self.register_buffer("pm_match_ready", torch.zeros(cfg.L))
elif _K > 0:
    self.pm_mu = nn.Parameter(torch.randn(_K, cfg.d))           # |mu| ~ sqrt(d), LN scale
    self.pm_log_kappa2 = nn.Parameter(
        torch.full((_K,), -math.log(cfg.d)))                   # exponent |h - mu|^2 / d
if _K > 0:
    _hl = torch.logspace(
        math.log10(getattr(cfg, "poisson_halflife_min", 4.0)),
```

```
        math.log10(getattr(cfg, "poisson_halflife_max", 128.0)), _K)
    _lam = 0.5 ** (1.0 / _hl)
    self.pm_logit_lambda = nn.Parameter(torch.log(_lam / (1 - _lam)))
    self.pm_depth = nn.Parameter(torch.zeros(cfg.L, _K))
    self.register_buffer("pm_share", torch.zeros(cfg.L), persistent=False)
```

Two details matter later. The centres are drawn at the LayerNorm scale (`|mu| ~ sqrt(d)`), which is the scale of layer 1's input but not of layer 0's (see *Why the Trained Modes Cannot Match*). And one set of centres and widths serves both layers; only the depths are per layer.

The Occupation: poisson_mode_occupation

The overlap (pm.2) is computed from the expansion $\|h-\mu\|^2 = \|h\|^2 + \|\mu\|^2 - 2\,h\cdot\mu$, clamped at zero against round-off, in float32 with autocast disabled:

```
def poisson_mode_occupation(self, h: torch.Tensor, layer_idx: Optional[int] = None):
    """Mode overlaps E (B, T, K) and strict-past occupations phi (B, T, K).

    E[b, s, v] = exp(-kappa_v^2 |h_s - mu_v|^2) is token s's overlap with
    mode v (the single-particle note's overlap identity) and its creation
    rate. phi[b, t, v] = sum_{s<t} lambda_v^(t-1-s) E[b, s, v] is the
    Poisson mean of the immigration-death process: Poisson(E) particles
    enter per token, each survives a token with probability lambda_v.
    With cfg.poisson_match_rank > 0 the overlap is the layer's low-rank one
    (_pm_match) and layer_idx is required.
    """
    B, T, d = h.shape
    with torch.autocast(device_type=h.device.type, enabled=False):
        if getattr(self.cfg, "poisson_match_rank", 0):
            if layer_idx is None:
                raise ValueError("poisson_match_rank > 0: poisson_mode_occupation needs layer_idx")
            E = self._pm_match(h, layer_idx)[0]
        else:
            hf = h.float()
            mu = self.pm_mu.float()
            k2 = self.pm_log_kappa2.float().exp()                    # (K,)
            d2 = ((hf * hf).sum(-1, keepdim=True) + (mu * mu).sum(-1)
                  - 2.0 * hf @ mu.t()).clamp_min(0.0)                # (B, T, K)
            E = torch.exp(-k2 * d2)
        phi = self._pm_occupation_from_overlap(E)
    return E, phi
```

The occupation (pm.3) is computed as a matrix product rather than by the recursion: a (K,T,T) tensor of decay factors λ_v^{t-1-s}, masked to $s<t$, contracted with the overlaps. This is parallel over positions and exact; the decay tensor holds KT^2 floats, shared by the batch: about 17 MB at $K=16$ and $T=512$, 67 MB at $K=64$:

```
def _pm_occupation_from_overlap(self, E: torch.Tensor) -> torch.Tensor:
    """phi[b, t, v] = sum_{s<t} lambda_v^(t-1-s) E[b, s, v], float32 (B, T, K)."""
    T = E.shape[1]
    log_lam = torch.nn.functional.logsigmoid(self.pm_logit_lambda.float())
    t = torch.arange(T, device=E.device)
    lag = (t[:, None] - t[None, :] - 1)                              # t - 1 - s
    past = lag >= 0                                                  # s < t
    Lam = torch.where(past, torch.exp(lag.clamp_min(0)[None].float()
                                      * log_lam[:, None, None]),
                      torch.zeros((), device=E.device))              # (K, T, T)
    return torch.einsum("kts,bsk->btk", Lam, E)
```

`logsigmoid` gives $\ln\lambda_v$ directly from the logit, so λ_v stays in $(0,1)$ without clipping.

The Force: poisson_mode_force

The force (pm.5) is evaluated in closed form, with no autograd: $f = -\left(\left(\sum_v w_v\right)h - \sum_v w_v\,\mu_v\right)$, which is `-(w.sum(-1) * h - w @ mu)`:

```
def poisson_mode_force(self, h: torch.Tensor, layer_idx: int) -> torch.Tensor:
    """-grad_{h_t} U, U = -sum_v phi_v(t) a_v exp(-kappa_v^2 |h_t - mu_v|^2).

    phi uses tokens s < t only, so it is constant in h_t and this is the
    exact gradient in h_t; the loss still reaches earlier tokens through
    phi (live, the Gen 3 convention). a_v is pm_effective_depth.
    With cfg.poisson_match_rank > 0, |h - mu_v|^2 is |P_l (h - mu_{l,v})|^2
    (_pm_match), so -grad U = -P_l^T sum_v w_v P_l (h - mu_{l,v}).
    """
    if getattr(self.cfg, "poisson_match_rank", 0):
        with torch.autocast(device_type=h.device.type, enabled=False):
            E, z, m, k2 = self._pm_match(h, layer_idx)
            phi = self._pm_occupation_from_overlap(E)
            _ctx = getattr(self, "_flow_ctx", None)
            if _ctx is not None:
                phi = _ctx.setdefault("phi", phi)
            w = phi * E * (2.0 * k2 * self.pm_effective_depth(layer_idx))    # (B, T, K)
            fz = -(w.sum(-1, keepdim=True) * z - w @ m)                      # (B, T, r)
            f = fz @ self.pm_P[int(layer_idx)].float()                       # (B, T, d)
        return f.to(h.dtype)
    E, phi = self.poisson_mode_occupation(h)
    _ctx = getattr(self, "_flow_ctx", None)
    if _ctx is not None:                    # per-layer flow: phi from the layer's first substep
        phi = _ctx.setdefault("phi", phi)
    with torch.autocast(device_type=h.device.type, enabled=False):
        k2 = self.pm_log_kappa2.float().exp()
        a = self.pm_effective_depth(layer_idx)
        w = phi * E * (2.0 * k2 * a)                                  # (B, T, K)
        hf = h.float()
        f = -(w.sum(-1, keepdim=True) * hf - w @ self.pm_mu.float())
    return f.to(h.dtype)
```

The `_flow_ctx` branch serves the per-layer flow test (FLOW-R): when a layer is refined into several substeps, the occupations are frozen at the layer's first substep, as the trained step freezes them. The first branch is the content-selective matcher (*Implementation: poisson_match_rank*).

The effective depth applies the optional cap:

```
def pm_effective_depth(self, layer_idx: int) -> torch.Tensor:
    """The well depths a_v the force uses at this layer (K,), float32.

    The raw parameter pm_depth unless poisson_depth_cap is set, in which
    case cap * tanh(pm_depth / cap). Diagnostics must read this, not
    pm_depth, so that a capped model reports the depths it acts with.
    """
    a = self.pm_depth[layer_idx].float()
    cap = getattr(self.cfg, "poisson_depth_cap", None)
    if cap is not None:
        a = cap * torch.tanh(a / cap)
    return a
```

Where the Force Enters: _layer_forces

`_layer_forces` returns the conservative force at a point. The wells' force is added to the pair force f_ϕ, and its size relative to the rest is recorded per layer (`pm_share`, logged during training):

```
# PM1: the Poisson-mode force, a plain kick like the exchange field.
if self.pm_mu is not None:
```

```
    _f_pm = self.poisson_mode_force(h_in, layer_idx)
    with torch.no_grad():
        _cons = f_phi if f_theta is None else f_theta + f_phi
        self.pm_share[layer_idx] = _f_pm.norm() / (_cons.norm() + 1e-12)
    f_phi = f_phi + _f_pm
```

Under PM1 this is the whole story: the kick carries the wells. Under PMX the exact flow carries their quadratic and the kick subtracts what the flow already integrated (below).

The Wells' Quadratic: poisson_mode_quadratic

PMX needs, at the step's start, the force and the Hessian pieces of (pm.6). The rank part is returned only for the token's top wells with the largest force $|w_v|\,\|r_v\|$, because the batched eigensolvers on the GPU stop at 32×32 and V_θ already uses up to 16 of those dimensions:

```
def poisson_mode_quadratic(self, h: torch.Tensor, layer_idx: int, top: Optional[int] = None):
    """PMX: the wells' force and Hessian pieces at h, with phi from h's context.

    Returns (F, alpha, r, dW), float32. F = -grad U at h over every well,
    exactly :meth:`poisson_mode_force`'s value, and alpha (B, T, 1) the
    isotropic curvature of every well. The rank part is returned for the
    token's ``top`` wells with the largest force |w_v| |r_v| (all K if None):
    r (B, T, M, d) with r_v = h - mu_v, and dW (B, T, M), so that the
    Hessian of those wells' share is alpha I - sum_v dW_v r_v r_v^T. Live
    tensors: the caller detaches what goes into the exact flow's spring.
    """
    if getattr(self.cfg, "poisson_match_rank", 0):
        raise NotImplementedError("poisson_mode_quadratic (PMX) does not support poisson_match_rank
> 0")
    E, phi = self.poisson_mode_occupation(h)
    with torch.autocast(device_type=h.device.type, enabled=False):
        k2 = self.pm_log_kappa2.float().exp()
        a = self.pm_effective_depth(layer_idx)
        mu = self.pm_mu.float()
        w = phi * E * (2.0 * k2 * a)                                  # (B, T, K)
        hf = h.float()
        F = -(w.sum(-1, keepdim=True) * hf - w @ mu)
        alpha = w.sum(-1, keepdim=True)                               # (B, T, 1)
        K = w.shape[-1]
        if top is None or int(top) >= K:
            r = hf.unsqueeze(-2) - mu                                 # (B, T, K, d)
            dW = 2.0 * k2 * w
        else:
            d2 = ((hf * hf).sum(-1, keepdim=True) + (mu * mu).sum(-1)
                  - 2.0 * hf @ mu.t()).clamp_min(0.0)                 # |r_v|^2, (B, T, K)
            idx = (w.abs() * d2.sqrt()).topk(int(top), dim=-1).indices   # (B, T, M)
            r = hf.unsqueeze(-2) - mu[idx]                            # (B, T, M, d)
            dW = 2.0 * k2[idx] * w.gather(-1, idx)                    # (B, T, M)
    return F, alpha, r, dW
```

The PMX Layer Step

In _layer_step_langevin the wells' quadratic joins V_θ's retained modes. indefinite_lowrank_modes_chol builds an orthonormal basis of $\mathrm{span}[U_\theta, r_v]$ and the eigenpairs of the combined spring on it, with signs $+1$ for V_θ's modes and $-\mathrm{d}W_v$ for the wells. _pm_quad_force is the linearised wells' force the flow carries, $F(h_0) - H(h - h_0)$:

```
if pmx:
    # PMX: the wells' quadratic at the step's start (in the
    # per-layer flow, at the layer's first substep) joins Vtheta's
    # low-rank part: K = alpha I + modes of G G^T - W on
    # span[G, r_v], integrated exactly with damping on all of R^d.
    if flow_ctx is not None and "pmq" in flow_ctx:
        pm_h0, pm_F0, pm_alpha, pm_r, pm_dW = flow_ctx["pmq"]
```

```
    else:
        pm_F0, pm_alpha, pm_r, pm_dW = self.poisson_mode_quadratic(
            h_in, layer_idx, top=getattr(cfg, "poisson_wells_exact_modes", 16))
        pm_h0 = h_in
        if flow_ctx is not None:
            flow_ctx["pmq"] = (pm_h0, pm_F0, pm_alpha, pm_r, pm_dW)
    # Vtheta's retained modes exactly as SR2 keeps them; their
    # projected force goes into the flow, the soft modes stay in the kick
    th_U, th_kappa = lowrank_modes(
        lr_G, max_modes=getattr(cfg, "lowrank_max_modes", None),
        niter=getattr(cfg, "lowrank_niter", 2),
        oversample=getattr(cfg, "lowrank_oversample", 4),
        driver=getattr(cfg, "lowrank_driver", "svd"),
    )
    # the same operator as indefinite_lowrank_modes on
    # B = [th_U sqrt(th_kappa), r_v], signs [+1, -dW], with the basis
    # built from th_U (already orthonormal): a 16-wide Gram solve, not 32
    # 2026-10-11: the well basis by Cholesky QR (indefinite_lowrank_modes_chol),
    # one batched SVD fewer than the split routine; same operator and span
    lr_U, lr_kappa = indefinite_lowrank_modes_chol(
        th_U, th_kappa, pm_r.transpose(-1, -2).to(lr_G.dtype), pm_dW.to(lr_G.dtype))
    # the damped-mode series needs fewer terms at PMX's gamma * dt/2 (host float)
    pm_nterms = series_terms_for(float(getattr(cfg, "fixed_gamma", 0.0) or 0.0) * half)

    def _th_proj(f):
        return torch.einsum('...dq,...q->...d', th_U,
                            torch.einsum('...dq,...d->...q', th_U, f))
    pm_alpha_s = pm_alpha.detach().to(h_in.dtype)

    def _pm_quad_force(hh):
        # F_w(h0) - H_w (hh - h0), live: the wells force the exact flow carries
        dh = (hh - pm_h0).float()
        Hdh = pm_alpha * dh - (pm_dW.unsqueeze(-1) * pm_r
                               * (pm_r * dh.unsqueeze(-2)).sum(-1, keepdim=True)).sum(-2)
        return (pm_F0 - Hdh).to(hh.dtype)
```

The A substep then calls the exact damped flow with the combined spring and the force at the substep's start:

```
h_mid, v_mid = lowrank_iso_damped_substep(
    h_in, v, lr_U, lr_kappa, pm_alpha_s, _th_proj(f_L) + _pm_quad_force(h_in),
    m_b, gamma, half, pm_nterms,
```

and the kick between the two A substeps removes the part of the force the flow already carries, so nothing is counted twice:

```
# the exact flow carries Vtheta's retained-mode force and the
# wells' quadratic in full; the soft modes stay in the kick
f_kick = f_kick - (_th_proj(f_L_mid) + _pm_quad_force(h_mid))
```

The Exact Substeps in cfc_baoab.py

The damped substep projects the velocity and force onto $\mathrm{span}(U)$ and its complement, and applies to each the closed-form flow of a damped oscillator with the corresponding stiffness (Appendix A2) [6]:

```
def lowrank_iso_damped_substep(
    h: torch.Tensor,
    v: torch.Tensor,
    U: torch.Tensor,
    kappa: torch.Tensor,
    alpha: torch.Tensor,
    f: torch.Tensor,
    m: torch.Tensor,
    gamma: torch.Tensor | float,
```

```
    dt: float,
    n_terms: int = 40,
) -> Tuple[torch.Tensor, torch.Tensor]:
    """Exact flow over ``dt`` of  m x'' = f - K (x - h) - gamma m x',  x(0) = h, x'(0) = v,

    with the frozen spring ``K = alpha I + U diag(kappa) U^T`` on ALL of R^d:
    modes in span(U) have stiffness ``kappa + alpha`` (either sign), the
    complement has ``alpha`` (per token, either sign), and the damping is
    integrated exactly everywhere. ``f`` is the force at ``h`` (the start of
    the substep), as in :func:`lowrank_damped_substep`. The caller must apply
    no O-step at all: friction is inside this flow on the whole space.
    ``alpha`` is (..., 1); ``U`` has orthonormal or zero columns.
    """
    vz = torch.einsum('...dq,...d->...q', U, v)
    fz = torch.einsum('...dq,...d->...q', U, f)
    vc = v - torch.einsum('...dq,...q->...d', U, vz)
    fc = f - torch.einsum('...dq,...q->...d', U, fz)
    E12s, E22s, Ps, Qs = (c.to(h.dtype) for c in damped_mode_coefficients((kappa + alpha) / m,
gamma, dt, n_terms))
    E12c, E22c, Pc, Qc = (c.to(h.dtype) for c in damped_mode_coefficients(alpha / m, gamma, dt,
n_terms))
    dz = E12s * vz + Qs * fz / m
    vz_new = E22s * vz + Ps * fz / m
    dc = E12c * vc + Qc * fc / m
    vc_new = E22c * vc + Pc * fc / m
    h_new = h + torch.einsum('...dq,...q->...d', U, dz) + dc
    v_new = torch.einsum('...dq,...q->...d', U, vz_new) + vc_new
    return h_new, v_new
```

The first-order counterpart moves each direction by $\mu_{\mathrm{FO}}\Delta t\,\varphi(\mu_{\mathrm{FO}}k\Delta t)$ times its force component, $\varphi(x) = (1 - e^{-x})/x$ (Appendix A3):

```
def lowrank_iso_overdamped_substep(
    h: torch.Tensor,
    U: torch.Tensor,
    kappa: torch.Tensor,
    alpha: torch.Tensor,
    f: torch.Tensor,
    mu: torch.Tensor | float,
    dt: float,
) -> torch.Tensor:
    """Exact gradient flow over ``dt`` of  x' = mu (f - K (x - h)),  x(0) = h.

    FO-b on PMX (protocol SS5.13, 2026-10-19): the first-order counterpart of
    :func:`lowrank_iso_damped_substep`, with the same frozen spring
    ``K = alpha I + U diag(kappa) U^T`` on all of R^d: modes in span(U) relax
    with stiffness ``kappa + alpha``, the complement with ``alpha`` (per token;
    either sign: a negative stiffness grows as e^(mu |k| dt)). ``f`` is the
    force at ``h``. Each direction moves by mu dt phi(mu k dt) times its force
    component, phi(x) = (1 - e^-x) / x. Returns ``h_new``.
    """
    fz = torch.einsum('...dq,...d->...q', U, f)
    fc = f - torch.einsum('...dq,...q->...d', U, fz)
    dz = (mu * dt) * _relax_coeff(mu * (kappa + alpha) * dt) * fz
    dc = (mu * dt) * _relax_coeff(mu * alpha * dt) * fc
    return h + torch.einsum('...dq,...q->...d', U, dz) + dc
```

and the FO-b layer step is A–B–A: exact relaxation over $\Delta t/2$, one gradient step on everything the relaxation does not integrate, relaxation again, and zero velocity into the next layer:

```
def _overdamped_step(self, h_in, xis, vtheta_comps, layer_idx, dt, m_b,
                     lr_U, lr_kappa, alpha, a_force, kick_sub, flow_ctx):
    """FO-b (cfg.first_order_overdamped): one A-B-A step of the gradient flow.
```

```
    A: exact relaxation over dt/2 under the frozen spring the second-order
       step's A substeps integrate: the retained low-rank modes (SR2), or
       with PMX ``alpha I + modes`` on all of R^d (alpha not None);
    B: gradient step (mu dt) f on everything A does not integrate, at h_mid;
    A: relaxation again, from the moved point. mu = dt_trained / (2 m).
    B is one explicit (Euler) step of its part of the flow, so the scheme is
    first order in dt, as any first-order step with one force evaluation is;
    the per-layer flow (k substeps) converges to the gradient flow as 1/k
    (debug/verify_fob_switch.py). Returns ``(h_new, h_new)``: zero velocity
    into the next layer.
    """
    cfg = self.cfg
    half = 0.5 * dt
    mu = getattr(self, "_fob_dt", cfg.dt) / (2.0 * m_b)

    def relax(hh):
        if alpha is None:
            return lowrank_overdamped_substep(hh, lr_U, lr_kappa, a_force(hh), mu, half)
        return lowrank_iso_overdamped_substep(hh, lr_U, lr_kappa, alpha, a_force(hh), mu, half)
    h_mid = relax(h_in)
    if not h_mid.requires_grad:
        h_mid = h_mid.requires_grad_(True)
    f_theta, f_phi = self._layer_forces(
        h_mid, xis, layer_idx, split=True, vtheta_comps=vtheta_comps,
    )
    f_kick = f_theta + f_phi - kick_sub(h_mid)
    if cfg.force_clamp_max is not None:
        f_kick = f_kick.clamp(-cfg.force_clamp_max, cfg.force_clamp_max)
    h_mid = h_mid + (mu * dt) * f_kick
    h_new = relax(h_mid)
    if cfg.ln_after_step and (flow_ctx is None or flow_ctx.get("project", True)):
        h_new = self._project(h_new)
    return h_new, h_new
```

The Notebook: Switches, Tags and Guards

Every switch in Cell 0 is off by default, and every non-default value adds a part to the run's tag, which names its Drive folder. A run with a new setting therefore never resumes another run's checkpoint:

```
POISSON_MODES        = 0
POISSON_MODE_CLIP    = 0.3
# PM1-cap (protocol SS5.15, 2026-10-08): bound every well depth smoothly,
# a = cap * tanh(a_raw / cap). The localization showed PM1's refinement failure
# and momentum dependence come from the strength of the layer-1 wells; weakening
# them post hoc to half strength brought Gate 3 at N = 4 from +19,029% to +133%.
# A reparameterisation only: the force stays an exact gradient. None = off
# (bit-identical to PM1). Tag 'pmcap<cap>'.
POISSON_DEPTH_CAP    = None
# PMX (protocol SS5.15, 2026-10-10): integrate the wells exactly. Their
# quadratic at each step's start (phi fixed) joins Vtheta's low-rank part in the
# exact damped flow over all of R^d; the kick keeps only the remainder. Needs
# POISSON_MODES > 0 and LOWRANK_DAMPED_FLOW. False = off (bit-identical).
# Every well's force and isotropic curvature go into the flow; the rank part of
# the token's POISSON_WELLS_EXACT_MODES strongest wells joins the eigensolve
# (13% of PM1's layer-1 wells force stays in the kick at 16). LOWRANK_MAX_MODES
# + this must be <= 32: the GPU's batched eigensolvers stop at 32 x 32.
# Tag 'pmx<modes>'.
POISSON_WELLS_EXACT  = False
POISSON_WELLS_EXACT_MODES = 16

# Content-selective Poisson modes (Stage 0', 2026-10-23; Implementing_Semantic_Templates_A_Path.md
# SS7). The default modes do not discriminate between tokens (protocol SS5.13: the occupations are
# 99.8% position, a content swap costs 0.02-0.09%): their centres sit outside layer 0's small state
```

```
# cloud, one set of centres serves two layers, and in d = 384 an isotropic Gaussian cannot grade.
# POISSON_MATCH_RANK = r > 0 matches in a learned rank-r subspace per layer, with per-layer
# centres and a width that follows the cloud, all set from the layer's own token states on the
# first training step (debug/narrow_wells_design.py: r = 8 reaches 6-15% of tokens per mode with
# 94-99% of tokens covered). Explicit kick only: needs POISSON_WELLS_EXACT = False.
# 0 = off (bit-identical). Tag 'pmlr<r>'.
POISSON_MATCH_RANK    = 0
# The modes' initial half-lives, log-spaced over [MIN, MAX] tokens (learned afterwards). An
# occupation averages a mode's overlaps over its half-life window, which washes content out: with
# the rank-8 matcher the occupations are 94-96% position at 4-128 and 41-43% at 1-16
# (debug/pm_halflife_range.py); trained models moved 4-128 to about 4-16 on their own (PMX-K16:
# p05 4.2, p50 7.9, p95 15.7). Defaults 4.0 / 128.0 (bit-identical). Tag 'pmhl<min>t<max>'
otherwise.
POISSON_HALFLIFE_MIN = 4.0
POISSON_HALFLIFE_MAX = 128.0


if POISSON_MODES:
    _variant_parts.append(f'pm{POISSON_MODES}')
    if (POISSON_HALFLIFE_MIN, POISSON_HALFLIFE_MAX) != (4.0, 128.0):
        assert 0 < POISSON_HALFLIFE_MIN <= POISSON_HALFLIFE_MAX, 'POISSON_HALFLIFE_MIN/MAX: need 0 <
MIN <= MAX'
        _variant_parts.append(f'pmhl{POISSON_HALFLIFE_MIN:g}t{POISSON_HALFLIFE_MAX:g}'.replace('.',
'p'))
    # 2026-10-06: conditional, so the 0.3 probe's tag is unchanged. Without it a
    # different clip would resume the 0.3 arm's folder as if it were the same run.
    if abs(POISSON_MODE_CLIP - 0.3) > 1e-12:
        _variant_parts.append(f'pmclip{POISSON_MODE_CLIP:g}'.replace('.', 'p'))
    if POISSON_DEPTH_CAP is not None:
        _variant_parts.append(f'pmcap{POISSON_DEPTH_CAP:g}'.replace('.', 'p'))
    if POISSON_WELLS_EXACT:
        _variant_parts.append(f'pmx{POISSON_WELLS_EXACT_MODES}')
    if POISSON_MATCH_RANK:
        assert not POISSON_WELLS_EXACT, 'POISSON_MATCH_RANK runs on the explicit kick: set
POISSON_WELLS_EXACT = False'
        _variant_parts.append(f'pmlr{POISSON_MATCH_RANK}')
```

Cell 5 passes the switches to the configuration (older Cell 0 versions without the newest switches still run, through `globals().get`), and Cell 5b refuses to train a model whose tag or built configuration disagrees with Cell 0:

```
_pmhl = (float(globals().get('POISSON_HALFLIFE_MIN', 4.0)),
float(globals().get('POISSON_HALFLIFE_MAX', 128.0)))
_pmhl_tag = f'pmhl{_pmhl[0]:g}t{_pmhl[1]:g}'.replace('.', 'p')
if _pmhl != (4.0, 128.0):
    assert _pmhl_tag in _variant_tag.split('_'), (
        'POISSON_HALFLIFE_MIN/MAX is missing from the variant tag -- this run would share '
        'the default-range arm\'s Drive folder and resume from its checkpoint.')
    assert (getattr(model.cfg, 'poisson_halflife_min', 4.0), getattr(model.cfg,
'poisson_halflife_max', 128.0)) == _pmhl, (
        'Cell 0 asks for another half-life range but the built model ignores it (git pull).')
else:
    assert not any(t.startswith('pmhl') for t in _variant_tag.split('_')), 'pmhl in the tag with the
default half-life range'
_pmlr = int(globals().get('POISSON_MATCH_RANK', 0))
if _pmlr:
    assert f'pmlr{_pmlr}' in _variant_tag.split('_'), (
        'POISSON_MATCH_RANK is missing from the variant tag -- this run would share '
        'the isotropic-wells arm\'s Drive folder and resume from its checkpoint.')
    assert getattr(model.cfg, 'poisson_match_rank', 0) == _pmlr and getattr(model, 'pm_P', None) is
not None, (
        'Cell 0 asks for content-selective modes but the built model ignores it -- '
        'the repo model files predate it (git pull).')
else:
    assert not any(t.startswith('pmlr') for t in _variant_tag.split('_')), 'pmlr in the tag with no
POISSON_MATCH_RANK'
```

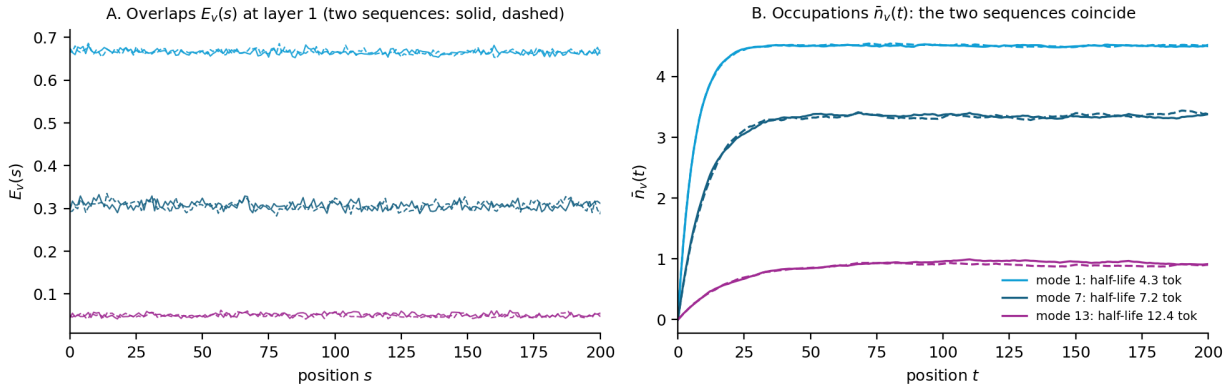
```
assert getattr(model, 'pm_mu', None) is not None and model.pm_mu.shape[-2] == POISSON_MODES, (
    'Cell 0 asks for Poisson modes but the built model has none -- the '
    'repo model files predate PM1 (git pull).')
```

The wells' parameters have their own gradient-clipping group: every parameter whose name starts with `pm_` is clipped at `POISSON_MODE_CLIP` (0.3), separately from the default group (1.0).

What the Trained Wells Do

The measurements below were made on the best checkpoints of three arms at 0.53B tokens: PMX ($K = 64$, exact wells, second order), FO-b-PMX (its first-order twin) and PMX-K16 ($K = 16$). The tools are listed in *The Diagnostic Tools* below.

Position, Not Content



PMX-K16 at layer 1: the overlaps of three modes and their occupations along two different sequences.

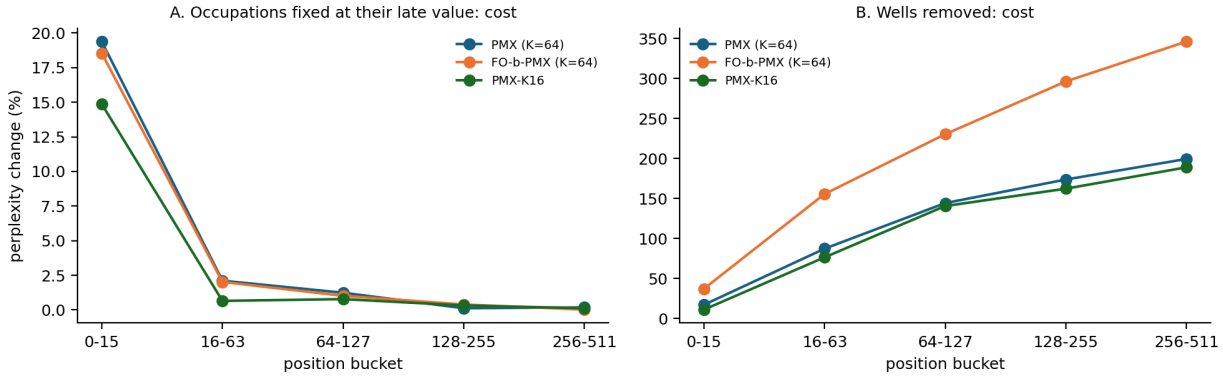
The trained modes barely discriminate between tokens. A mode's overlap varies by 1–4% from token to token (selectivity 0.01–0.04), so its occupation is close to the deterministic curve

$$\bar{n}_\nu(t) \approx \bar{E}_\nu \, \frac{1-\lambda_\nu^{t-1}}{1-\lambda_\nu} \qquad \text{(pm.9)}$$

with $\bar{E}_\nu$ the mode's average overlap: a function of the position alone. The figure shows it: two different texts give occupation curves that coincide. Rebuilt from position alone, the wells' force reproduces 99.8–99.98% of its variance, and giving a sequence another text's occupations changes perplexity by 0.02–0.09%, while removing the wells costs 155–282%. The models rely on the wells, but not on any memory of content.

The model has no positional encoding, and the context channels ξ are normalised averages, which carry almost no position signal. The un-normalised occupations are its only explicit clock.

A Fixed Landscape Beyond the First Tokens



The cost of fixing every occupation at its late value (A) and of removing the wells (B), by position.

An occupation rises over a few half-lives and then levels off (trained half-lives are 3–20 tokens). Replacing every occupation by one position-independent value, its mean over positions 256–511, costs about 1% overall: 15–19% in the first 16 tokens, about 2% up to position 63, and nothing from position 128 on. Removing the wells, by contrast, costs more the later the position. So, at 0.53B tokens, the wells act mainly as a **fixed extra potential**, K Gaussian wells per layer with learned centres, widths and per-layer depths added to V_θ's context-driven wells, with a start-of-sequence correction on top (Appendix A7).

Sixteen Modes Against Sixty-Four

reading	PMX-K16	PMX ($K = 64$)	FO-b-PMX ($K = 64$)
settled perplexity	54.24	54.63	54.02
velocity reset (Gate 1)	+5,788%	+91%	0 by construction
stiffest-mode $\omega\Delta t$, median	3.65	2.65	
attractive depths, layer 0 / layer 1	19% / 94%	50% / 48%	64% / 61%
N_{eff}, layer 0 / layer 1	11.0 / 13.6 of 16	39.8 / 36.7 of 64	48.4 / 39.1 of 64
force reproduced from position	99.96%	99.89%	99.82%

Sixteen modes give the second-order model the same perplexity as sixty-four, as a smooth positional potential needs few functions. The organisation differs: with fewer wells the model leans about sixty times as much on its velocity and its stiff rotation, and its wells split by layer (repulsive at layer 0, attractive at layer 1).

The Diagnostic Tools

All of them run on CPU from a downloaded run folder (`ckpt=` selects a snapshot, `folder=` another arm; K is read from the folder's `pm<K>` tag) [7]:

- `pm_mode_usage.py`: N_{eff}, N_{80}, per-token N_{eff}, selectivity, the overlap exponent, attractive shares, half-lives, and the position-only reconstruction.
- `pm_content_swap.py`: occupations from another sequence, averaged by position, or zero.
- `pm_static_landscape.py`: occupations fixed at their late value, everywhere or from $t = 128$, by position bucket.
- `mechanism_ablations.py`: the content paths (ξ swapped, its context channels only, V_ϕ off, the wells swapped or off).

- `vtheta_omega_dt.py`: V_θ's stiffest-mode $\omega\Delta t$ with one definition for both orders.
- `run_diag_cell_cpu.py`: any notebook diagnostic cell, unchanged, on CPU.

The hook each of them uses is one line, because every path that computes occupations goes through `_pm_occupation_from_overlap` (or `poisson_mode_occupation` in the isotropic design). The content swap, for example:

```
orig_occ = model.poisson_mode_occupation
MODE = {'v': 'base'}
def occ(h):
    E, phi = orig_occ(h)
    if MODE['v'] == 'swap':
        phi = phi.roll(1, dims=0)
    elif MODE['v'] == 'posmean':
        phi = phi.mean(0, keepdim=True).expand_as(phi)
    elif MODE['v'] == 'off':
        phi = torch.zeros_like(phi)
    return E, phi
model.poisson_mode_occupation = occ
```

Toward Semantic Templates

What a Template Needs

A semantic template pairs a Pattern Matching Structure with an Inference Structure (book Definition 8). In reaction form it is catalytic and non-consuming [8]:

$$\sum_v k_v\, A_v \to \sum_v (\,k_v + l_v)\, A_v \qquad \text{(pm.10)}$$

where A_v is a particle in mode v, k_v the number the template requires and l_v the number it produces. The bank of all templates, together with transport and destruction, is the complete Liouvillian of the dynamics, so a bank cannot be hard-coded without hard-coding the whole inference dynamics, including the prior: templates with trivial matchers reduce to spontaneous creation $\varnothing \to A_v$, whose stationary state is Poisson with mean c/d (creation over destruction rate).

Anatomy: Three Kinds of Parameter

Continuous, already differentiable. The hard firing threshold of the original template apparatus is the zero-temperature limit of a rate:

$$r_{\mathcal{T}} = r_0\, \sigma(\beta(\mathrm{BC} - \Theta)) \to r_0\, \mathbb{1}[\mathrm{BC} > \Theta] \quad (\beta \to \infty) \qquad \text{(pm.11)}$$

with BC the Bhattacharyya coefficient between the template's cloud and the local state. Made bilinear, the matcher's threshold is a binding radius in inflection radii:

$$\mathrm{BC} > \Theta \Leftrightarrow \frac{\rho}{x^*} < \sqrt{2\ln(1/\Theta)} \qquad \text{(pm.12)}$$

so learning Θ is learning how far a template reaches, with the same κ that sets a well. The price of sharpness is that the gradient that trains Θ and the centroid vanishes as β grows, so annealing β is the central training schedule.

Discrete, but relaxable. The stoichiometry (k, l) is combinatorial. Read as intensities over modes, $k, l \in \mathbb{R}^K_{\geq 0}$ (the relaxation the occupations already use: an intensity instead of a count), a template becomes a learned vector field on mode space, parameterised through softplus; sparsity is top-k routing over the bank (the straight-through Gumbel machinery the pair potential already uses), and low-rank (k, l) is a precondition for trainability.

Neither. The pairing of the matched structure with the created one is a correspondence, not a density, and has no formulation yet. A placement map $\mu_{\text{new}} = \mu_{\text{match}} + \delta_{\mathcal{T}}$ is the weak baseline to falsify first.

Tiers and Stages

Templates learned at training and fired at inference (Tier 1) are reachable; templates induced from context with no weight update (Tier 2) are the real target. The staging of the implementing note [8]: Decision (2026-10-31): the first implementation is Tier 1 -- the bank learned at training time and fired at inference, on the Stage 0" store below; Tier 2, the model creating templates it did not have during inference, is the follow-on, reached by extending the Tier 1 training algorithm once Tier 1 is validated.

- **Stage 0, make creation real in the registers — retired (2026-10-31):** an active fraction that varies, and a salience that behaves as an intensity. Withdrawn rather than repaired: the register pool is fixed and the register write is the match-and-edit form the MCS reduction forbids (The Particle Store: Making Allocation Real). Its DP1 probe survives as criterion 2 of Stage 0".
- **Stage 0', make matching real:** modes whose overlaps discriminate between tokens, and a model that uses them (*The Narrow-Well Deliberations and Experiments*).
- **Stage 0", make allocation real:** a write-once, block-causal store of created particles, so that firing can instantiate a degree of freedom instead of overwriting a slot or deepening a well. First build 2026-10-31 (The Particle Store: Making Allocation Real).
- **Stage 1:** one catalytic template $A_u \to A_u + A_v$, with the built-in check that unconditional templates reproduce the log-unigram bias. Fired into the Stage 0" store.
- **Stage 2:** a bank (Gumbel top-k, low-rank (k, l), placement map), tested on the Dyck falsifier F1.
- **Stage 3:** induction.

Modes as Matchers: Form Without Function

A trained Poisson mode has the form of a template's matching half:

a template has	a trained Poisson mode has
a pattern, a density over Σ	a centre μ_v and width κ_v: a Gaussian cloud
a trigger at a threshold Θ	the overlap $E_v(t)$ against a radius
a firing record	the occupation $\bar{n}_v(t)$, decaying with λ_v
slots (dummy particles)	none: one centre
an inference region that writes a structure	none: it deepens a well

As trained it does not have the function. A matcher must respond to some tokens and not others; the trained modes respond to every token about equally (*Position, Not Content*). That is why Stage 0' exists: before a bank of modes can serve as matchers, they have to be made selective, and the model has to use the selectivity.

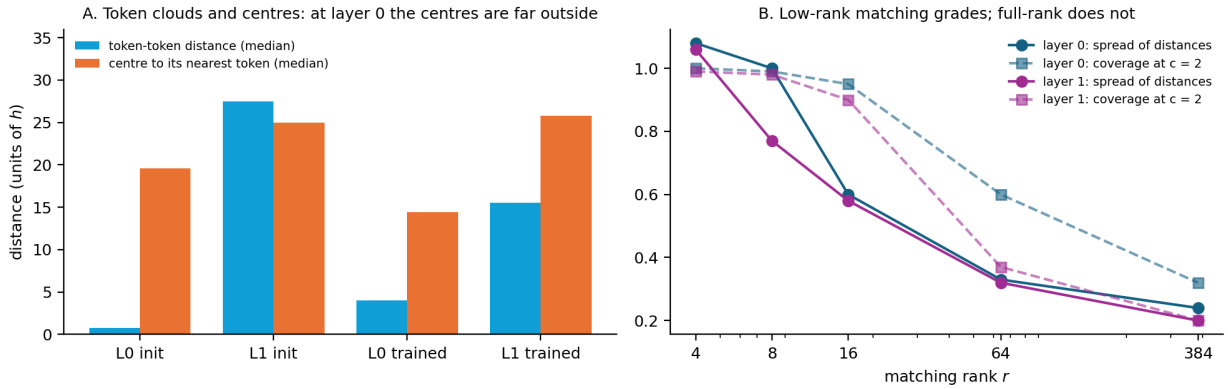
The Narrow-Well Deliberations and Experiments

Why the Trained Modes Cannot Match

The analysis `narrow_wells_design.py` measured the geometry of states and centres at initialisation and after training [9]. Three findings, each sufficient on its own:

1. **The centres sit outside the token cloud.** Layer 0's input is the token embedding: median norm 0.6 at initialisation and 2.9 trained, the whole cloud spanning 1–4 units. The centres sit at the LayerNorm scale, norm 19 at initialisation and 14 trained. Every token is then 14–19 units from every centre, the same distance to within 0.1–0.4. No width makes such a mode selective, and training did not move the centres in.
2. **One set of centres cannot serve two layers.** Layer 1's input lies on the LayerNorm sphere (norm 19.6), far from layer 0's small cloud near the origin.

3. **In 384 dimensions an isotropic Gaussian cannot grade.** Even with centres placed on trained token states, the token-to-centre distances spread by only 20–24% of their median (Appendix A4). Halving the width moves a mode from reaching 97–100% of tokens to reaching 1–6%.



Where the centres sit against the token clouds (A), and the spread of distances and the coverage against matching rank (B).

Low-Rank Matching

Measured in an r-dimensional subspace, the distances spread out. With $r = 8$ the spread is 0.77–1.00 of the median, a mode reaches a graded 6–15% of tokens, and 94–99% of tokens reach some mode. The design that follows matches each layer in its own learned subspace:

$$E_{\ell,v}(s) = \exp\left(-\kappa^2_{\ell,v}\left\|P_\ell(h_s - \mu_{\ell,v})\right\|^2\right), \qquad \kappa^2_{\ell,v} = \frac{c_{\ell,v}}{\bar{D}^2_\ell} \qquad \text{(pm.13)}$$

The potential is still a scalar function of h, so the force stays conservative; differentiating (pm.13),

$$f_{\text{pm}}(h_t) = -P_\ell^\top \textstyle\sum_v w_v\,(t)\, P_\ell(h_t - \mu_{\ell,v}), \qquad w_v(t) = 2\kappa^2_{\ell,v}\, a_{\ell,v}\, \bar{n}_v(t)\, E_{\ell,v}(t) \qquad \text{(pm.14)}$$

whose curvature lies in the r-dimensional span of $P_\ell^\top$. The width follows the cloud through the running median

$$\bar{D}^2_\ell \leftarrow \beta_D\, \bar{D}^2_\ell + (1-\beta_D)\,\text{median}_{s\neq s'}\|P_\ell(h_s - h_{s'})\|^2, \qquad \beta_D = 0.99 \qquad \text{(pm.15)}$$

because layer 0's cloud grows about fivefold in training, and a width fixed in absolute units would end too narrow.

Implementation: poisson_match_rank

`_pm_match` implements (pm.13). On the first training forward of each layer it sets the layer's projection, centres and width scale from the token states it receives, without gradient and without drawing from the global random stream; afterwards it updates (pm.15):

```
def _pm_match(self, h: torch.Tensor, layer_idx: int):
    """Content-selective overlap (cfg.poisson_match_rank > 0) at one layer.

    Returns E (B, T, K), z = h P_l^T (B, T, r), m = mu_l P_l^T (K, r) and
    kappa^2 (K,), float32. On the first training forward of each layer the
    layer's centres, projection and width scale are set from the token
    states it receives (P_l: their top-r principal directions; centres: the
    means of K k-means clusters in span(P_l); D2_l: the median squared
    projected distance between tokens); afterwards D2_l follows the cloud as
    a running median. Both are done without gradient and draw no global RNG.
    """
    l = int(layer_idx)
    hf = h.float()
    if self.training:
        with torch.no_grad():
            X = hf.detach().reshape(-1, hf.shape[-1])
```

```
            n = X.shape[0]
            if not bool(self.pm_match_ready[l]):
                g = torch.Generator(device="cpu").manual_seed(1234 + l)
                K, R = self.pm_mu.shape[1], self.pm_P.shape[1]
                sub = torch.randperm(n, generator=g)[:min(n, 8192)].to(X.device)
                Xs = X[sub]
                _, _, Vh = torch.linalg.svd(Xs - Xs.mean(0, keepdim=True), full_matrices=False)
                # Centres: k-means in the projected subspace (k-means++ seeding, 10 Lloyd passes),
                # each centre the mean of its cluster's states. Random samples leave ~15% of the
                # tokens unreached at K = 16 (minority regions get no centre).
                Zs = Xs @ Vh[:R].t()
                seeds = [int(torch.randint(len(Zs), (1,), generator=g))]
                dmin = ((Zs - Zs[seeds[0]]) ** 2).sum(-1)
                for _ in range(1, K):
                    pr = (dmin / dmin.sum().clamp_min(1e-30)).cpu()
                    seeds.append(int(torch.multinomial(pr, 1, generator=g)))
                    dmin = torch.minimum(dmin, ((Zs - Zs[seeds[-1]]) ** 2).sum(-1))
                C = Zs[seeds].clone()
                mu0 = Xs[seeds].clone()
                for _ in range(10):
                    a_ = ((Zs * Zs).sum(-1, keepdim=True) + (C * C).sum(-1) - 2.0 * Zs @
C.t()).argmin(-1)
                    for v in range(K):
                        sel = a_ == v
                        if bool(sel.any()):
                            C[v] = Zs[sel].mean(0)
                            mu0[v] = Xs[sel].mean(0)
                self.pm_mu.data[l].copy_(mu0)
                self.pm_P.data[l].copy_(Vh[:R])
                Z = Xs[:1024] @ Vh[:R].t()
                D2 = ((Z * Z).sum(-1, keepdim=True) + (Z * Z).sum(-1) - 2.0 * Z @
Z.t()).clamp_min(0.0)
                self.pm_D2[l] = D2[~torch.eye(len(Z), dtype=torch.bool, device=Z.device)].median()
                self.pm_match_ready[l] = 1.0
            else:
                Z = X[:: max(1, n // 1024)][:1024] @ self.pm_P[l].detach().float().t()
                D2 = ((Z * Z).sum(-1, keepdim=True) + (Z * Z).sum(-1) - 2.0 * Z @
Z.t()).clamp_min(0.0)
                cur = D2[~torch.eye(len(Z), dtype=torch.bool, device=Z.device)].median()
                a = float(getattr(self.cfg, "poisson_match_ema", 0.99))
                self.pm_D2[l] = a * self.pm_D2[l] + (1.0 - a) * cur
    P = self.pm_P[l].float()                                          # (r, d)
    z = hf @ P.t()                                                    # (B, T, r)
    m = self.pm_mu[l].float() @ P.t()                                 # (K, r)
    k2 = self.pm_log_c[l].float().exp() / self.pm_D2[l].float().clamp_min(1e-12)   # (K,)
    d2 = ((z * z).sum(-1, keepdim=True) + (m * m).sum(-1) - 2.0 * z @ m.t()).clamp_min(0.0)
    return torch.exp(-k2 * d2), z, m, k2
```

The force (pm.14) is the first branch of `poisson_mode_force` shown in *The Force: poisson_mode_force*: `fz = -(w.sum(-1) * z - w @ m)` in the projected coordinates, mapped back by `fz @ P`. The exact (PMX) path refuses the new mode explicitly (`poisson_mode_quadratic` raises), so the first trained test runs on the explicit kick.

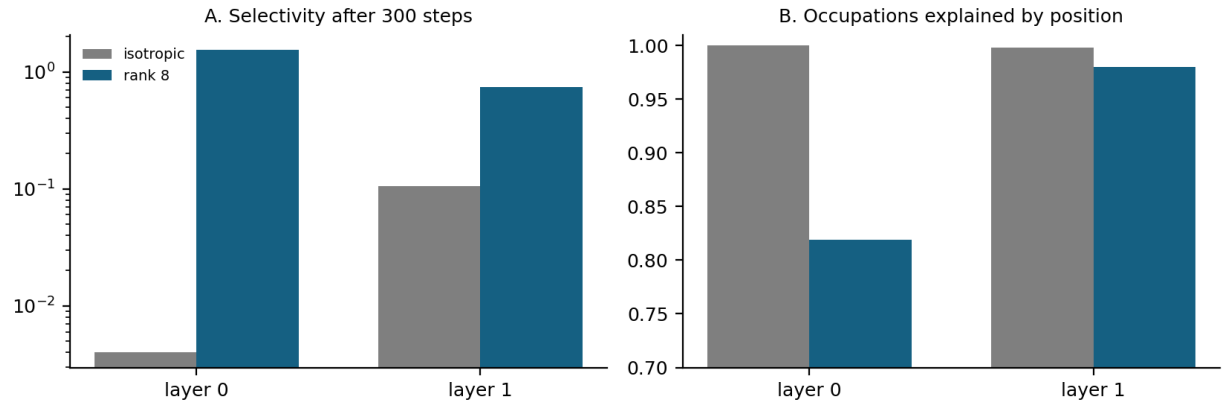
Two changes against the first design came from the checks. **Centres by k-means, not random samples:** at initialisation the projected cloud is a dense core holding about 75% of the tokens plus small tight clusters (most likely the frequent tokens' random embeddings), and 16 random samples left about 15% of the tokens unreached; k-means in the projected subspace reaches all of them (Appendix A6). **A starting width factor** $c = 1$**:** it keeps selectivity at 0.5 or more and coverage at 100% for both $K = 16$ and $K = 64$.

Verification

`verify_pm_match.py` checks the change in three modes [10]. *Fingerprint:* with the switch off, four existing configurations (F3.1, PM1, PMX-K16, FO-b-PMX-K16) built from the notebook's cells at seed 0 reproduce the

committed code exactly (loss, logits checksum, every parameter, every gradient, every tag), compared against a checkout of HEAD. *Checks*, with $r = 8$, $K = 16$, in both orders: the force equals $-\nabla_h V_{\text{pm}}$ by autograd to about 10^{-7}; selectivity at initialisation is 3.2–3.6 with 100% coverage and about five modes per token; every new parameter has a finite, non-zero gradient after one optimizer step; the running width moves in training and not in evaluation. *Planned:* the two configurations of the 2B template pair build from the notebook's cells and pass Cell 5b.

The Short Training Run

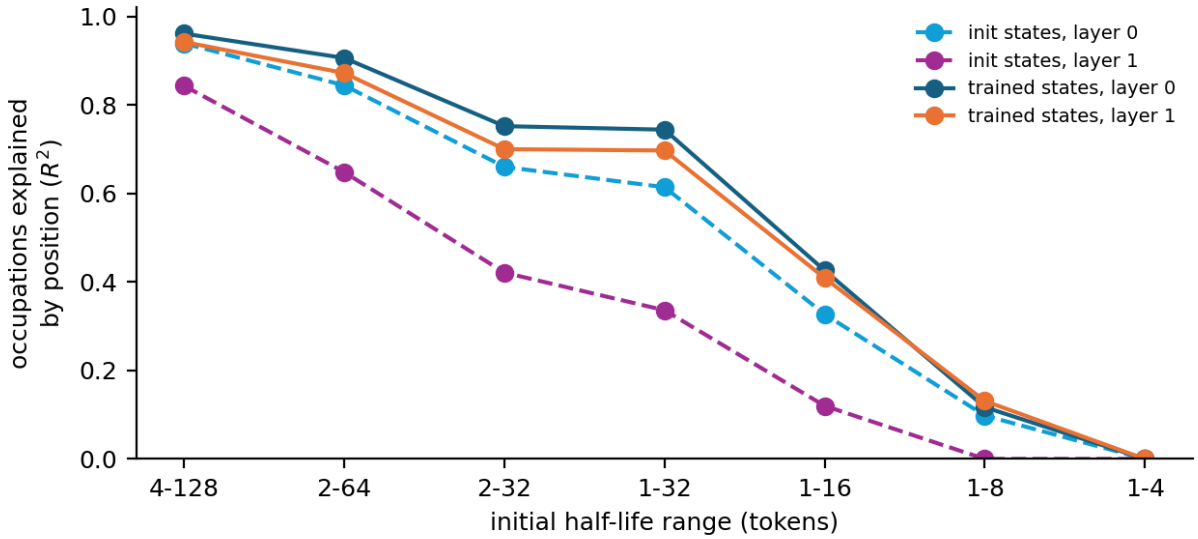


The short CPU run: selectivity (A) and the occupations' position share (B) after 300 steps.

Two arms trained from scratch on CPU for 300 steps of 4×256 tokens, identical except for the matcher (`pm_match_short_train.py`). After 300 steps the rank-8 selectivity is 1.54 at layer 0 and 0.74 at layer 1, against 0.004 and 0.11 for the isotropic modes; at layer 0 the occupations are 81.9% position (100.00% isotropic), at layer 1 98.0%; the content swap costs +0.08% (spread 0.20%), not yet a use of the content. The selectivity survives early training; whether a full run uses it is the trained test's question.

The Half-Life Range

Selective overlaps are necessary, not sufficient. An occupation sums a mode's overlaps over its half-life window, and the sum averages token-level variation away (Appendix A5). `pm_halflife_range.py` measured the position share by initial range:



The occupations' position share against the initial half-life range.

At the default 4–128 tokens the occupations stay 94–96% position even with selective overlaps; at 1–16 they are 41–43% position on the trained geometry, leaving 57–59% of their variance for content with a 16-token horizon. Trained models had already moved 4–128 to about 4–16 on their own (PMX-K16: p05 4.2, median 7.9, p95 15.7). The 2B template pair therefore uses 1–16.

The Particle Store: Making Allocation Real (Stage 0″)

The v0 expressivity ceiling rests on one hypothesis: "there is no mechanism by which the state space grows during inference (no creation, no destruction, no operator that maps to outside M)" [15]. Nothing else lifts it — not more parameters, not RL signal, not a richer force vocabulary — and §8.2 of the same note names what does: with creation in place "the simulator can grow its memory linearly with input length". A Poisson mode is a well: firing it deepens a potential and adds no degree of freedom (The Wells and Their Force). Templates lift the class only if firing instantiates a particle, and the first draft of the staging had no stage for that [8]. Stage 0" is that stage, added on 2026-10-31 together with a first build. This section records the design, the code, its verification, and what the build measures.

Why Neither Registers nor Modes Allocate

Three facts, each sufficient, say the slot registers cannot become the substrate however their salience is retrained:

1. **The pool is fixed.** `n_registers` is a constructor constant, and the per-position bank is (B, T, M, d): the state at position t is Md numbers whatever t is, so the hypothesis of Lemma 4 holds exactly. The book concedes it: "the class statement concerns the architecture family with a register pool that grows with the input, not the trained instances" [3].
2. **The write is the forbidden form.** The write port $r \leftarrow \sigma\, r + (1-\sigma)\,\mathrm{readout}$ matches against the active set and edits the matched slot in place. That is match-and-edit, under which the MCS reduction's polynomial parsing becomes PSPACE-complete recognition [16]. A template substrate has to consume and emit — destroy a reactant, bear a product elsewhere — and a slot that is rewritten in place cannot, whatever its salience does.
3. **DP1–DP3.** The trained registers are always occupied and their salience is not an intensity (From Slot Registers to Poisson Modes).

Stage 0 is therefore retired rather than repaired. The Poisson modes fail for the other reason: a mode creates particles at a fixed centre, so however many it creates the state it adds is one intensity per mode — K numbers. What grows the state is a particle whose payload is set at birth from the token that bore it, and which persists to be read, matched and consumed.

The Design: a Write-Once, Block-Causal Store

State. Per layer, slots (B, T, K, ·): at every position each of K particle types may bear one particle. A slot carries a bounded payload — a position in Σ, a type, a birth index — and an existence e ∈ [0, 1], the occupation a creation operator sets. In the first build the existence is the Stage 0' overlap of the token with the type, its Poisson creation mean, and the payload is the token state displaced by a learned per-type offset:

$$e_{s,j}^{(b)} = E_{b,j}\big(h_s^{(b)}\big), \quad \mu_{s,j}^{(b)} = h_s^{(b)} + \delta_j \text{(pm.16)}$$

for the particle of type j that token s bears at layer b. Capacity KT is preallocated, so allocation is realised as e leaving zero; what grows with the input is the number of live particles a token sees, up to Kt. **Payloads are written once and never again.** Consumption, when it arrives with the first template that has a reactant, thins the existence, $e_p \leftarrow e_p(1-\mathrm{fire})$, and touches nothing else: rebinding is destroy-old plus emit-new, never an edit.

Visibility and force. A particle born at (layer b, token s) is visible to every token t > s at every layer l ≥ b — a block-causal T × KT mask — and acts through the well potential the modes already use, evaluated in the acting layer's matching subspace:

$$U_\ell(h_t) = -\sum_{b\le\ell}\sum_{s<t}\sum_j a_{\ell,j}\, e_{s,j}^{(b)}\, \lambda_j^{t-1-s} \exp\left(-\kappa_{\ell,j}^2 \left\| P_\ell\left(h_t - \mu_{s,j}^{(b)}\right)\right\|^2\right) \quad \text{(pm.17)}$$

with a the layer's per-type depths (starting at 0, so the force is exactly zero at step 0), λ the per-type survival per token, and κ, P the acting layer's width and projection. The sources e and μ are of tokens s < t, so they are constant in h_t and the explicit gradient is exact in it; the loss still reaches the creating token through e and μ (live, the Gen 3 convention, as for the occupations in (pm.14)):

$$f_{\text{ps}}(h_t) = -\nabla_{h_t} U_\ell = -2\, P_\ell^{\top} \sum_{b,s,j} w_{t,s,j}^{(b)}\, \kappa_{\ell,j}^2\, P_\ell\left(h_t - \mu_{s,j}^{(b)}\right) \quad \text{(pm.18)}$$

where $w_{t,s,j}^{(b)}$ is the summand of (pm.17) without the sign. This is the additive v2 channel: order-blind by the commutativity theorem, and that is its job. The store is carried layer to layer; within a layer no particle depends on one born in the same layer, so a forward pass supports derivations of depth at most L — the depth floor of the implementing note again [8].

What the store satisfies, requirement by requirement:

requirement	how the store meets it
state grows with the input (the hypothesis of Lemma 4 fails)	live particles at t up to Kt; payloads persist
bounded multiplicity per firing	at most one particle per type per position, K in all
consume and emit, never match and edit	write-once payloads; consumption touches only the existence
bounded fragment dimension and arity	payload d + O(1); the two-slot stage reads two particles
finite bank	K fixed; the admission rule of Stage 2 governs growth
causal and parallel-trainable	block-causal mask; no particle-to-particle dependence within a layer
conservative token forces	wells in h; the force is the gradient by autograd (Verification of the Store)
exactness of the mean (The Occupation Is a Poisson Mean)	with no reactant the potential is linear in the existences, so carrying the mean is exact; with reactants the store is the object a sampled or mean-field closure acts on

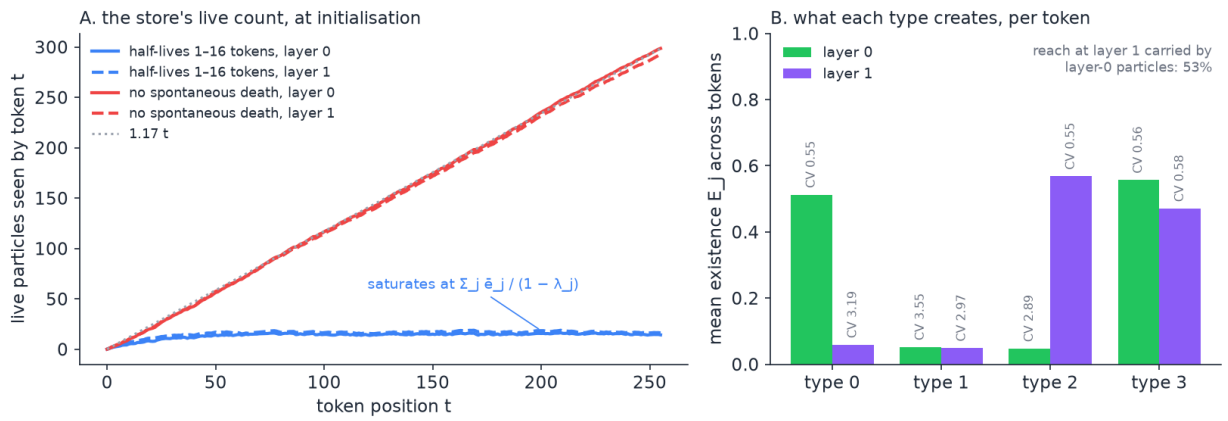
Cost: particle-to-token forces $O(KT^2d)$ per layer, attention over KT keys; memory (B, T, K, d) per layer. What the design does not change: consume-and-emit does not weaken the commutativity theorem. When two reactions both fire the resulting configuration is the same multiset in either order; consumption makes the second firing's rate path-dependent, not the result. The practical consequence is one the staging already books: a store with consumption can pass the let-binding falsifier F3 by destroy-plus-emit rebinding, and must then be described as such [8].

Decay Decides the Class

The number of live particles a token sees is the occupation summed over types,

$$N(t) = \sum_{s<t}\sum_j \lambda_j^{t-1-s}\, e_{s,j} \quad \longrightarrow \quad \sum_j \frac{\overline{e}_j}{1-\lambda_j} \quad (\lambda_j < 1), \quad N(t) \approx K\,\overline{e}\,t \quad (\lambda_j = 1) \quad \text{(pm.19)}$$

With spontaneous death (λ < 1) the count saturates — a decaying store is bounded memory however large the store, which is Lemma 2 of the bounds note (damping is information-destroying) in another guise [15]. With no spontaneous death the count grows with the input, and destruction is left to consumption. The first build exposes both as `PARTICLE_STORE_DECAY`. Measured at initialisation on 2 × 256 real tokens (K = 4, rank 8): with half-lives of 1–16 tokens, 11 / 15 / 14 particles at t = 32 / 128 / 255; without death, 35 / 148 / 298, about 1.2 t.



The store at initialisation: the live count a token sees with and without spontaneous death (A), and what each of the four types creates per token, with its coefficient of variation across tokens (B).

The class lift therefore needs λ = 1 with destruction only by consumption. The decaying arm is the bounded, safe variant for the first training runs; the no-decay arm is the one the expressivity argument is about.

Implementation: particle_store

The switch is `particle_store` = K (0 = off, bit-identical), with the matching rank, the width factor and its running median, the half-life range, the decay switch and the payload's gradient path [1]:

```
# Particle store (Stage 0'', 2026-10-31; Implementing_Semantic_Templates_A_Path.md
# SS7): a write-once, block-causal store of CREATED particles, the degree of freedom
# a template needs and neither a slot register (rewritten in place, fixed M) nor a
# Poisson mode (deepens a well at a fixed centre) supplies. K particle types. At
# every layer l each token s bears, for each type j, a particle with existence e =
# E_j(h_s) -- the Stage 0' rank-r overlap of the token with the type, its Poisson
# creation mean -- and payload mu = h_s + delta_j, written once. A particle born at
# (layer b, token s) acts on every token t > s at every layer l >= b through the
# well U = -sum a_{l,j} e lambda_j^(t-1-s) exp(-kappa_{l,j}^2 |P_l (h_t - mu)|^2),
# the explicit -grad_{h_t} U (strict past, so e and mu are constant in h_t); the
# loss reaches the creating token through e and mu (live, the Gen 3 convention). The
# number of live particles at token t grows as K t: the state is no longer O(1) in
# the input. Depths a start at 0 (force exactly 0 at step 0); half-lives lambda_j
# log-spaced over [HALFLIFE_MIN, HALFLIFE_MAX] tokens; matcher mu_l, P_l, D2_l set
# from the layer's states on the first training forward as in Stage 0'. One force
# evaluation per layer; explicit kick only. 0 = off (bit-identical). Tag
# 'ps<K>r<rank>'.
particle_store: int = 0
particle_store_rank: int = 8
particle_store_c_init: float = 1.0
particle_store_ema: float = 0.99
# the 1-16 range keeps content in the occupations (debug/pm_halflife_range.py)
particle_store_halflife_min: float = 1.0
particle_store_halflife_max: float = 16.0
# False: no spontaneous death (lambda = 1, no half-life parameter); a particle lives
# until a reaction consumes it, which is what lets the live count grow with the
# input (a decaying memory saturates at sum_j E_j / (1 - lambda_j), bounds note
# Lemma 2). Tag suffix 'nd'.
particle_store_decay: bool = True
# 'live' | 'detached': does the loss reach h_s through the payload mu?
particle_store_payload: str = "live"
```

Construction follows the Poisson modes: nothing is created unless enabled, nothing draws RNG, and the depths start at zero. The parameters are `pstore_mu` (L, K, d), `pstore_P` (L, r, d), `pstore_log_c` (L, K), `pstore_depth` (L, K), `pstore_delta` (K, d) and, with decay, `pstore_logit_lambda` (K,); the buffers `pstore_D2` and `pstore_ready`

are the matcher's running median and its initialisation flag, `pstore_share` and `pstore_stats` diagnostics. The matcher is the Stage 0' one: `_pm_match` now delegates to a generic `_lowrank_match(h, l, mu, P, log_c, D2, ready, ema, seed)` with the same operations in the same order (the modes seed their k-means at 1234 + l, the store at 4321 + l), and `_ps_match` calls it with the `pstore_*` tensors. The store itself is a dictionary on the model, birth layer → (existences (B, T, K), payload states (B, T, d)), emptied by `_ps_begin()` at the start of every pass and written only inside the stack loop (`_in_stack_forward`), so that a layer-checkpoint recompute in the backward reads the forward's record and records nothing. `_ps_pairs` builds the pair terms of (pm.17) between the tokens at the acting layer and every particle they see:

```
def _ps_pairs(self, h: torch.Tensor, layer_idx: int, detach_sources: bool = False):
    """Pair terms between the tokens at h (acting layer l) and every particle they see.

    Returns z = h P_l^T (B, T, r), kappa^2 (K,) and a list over birth layers
    b <= l of (w, wu, c): w (B, T, T, K), indexed [t, s, j], the weight
    a_{l,j} e_b[s, j] lambda_j^(t-1-s) 1[s < t] exp(-kappa_j^2 |z_t - c_{s,j}|^2)
    of the particle of type j that token s bore at layer b; wu the same
    without the depth a (the reach, for the readings); and c (B, T, K, r)
    its projected payload P_l (h_b[s] + delta_j). The potential is -sum w.
    This layer's own particles (e = E_j(h), payload h) are computed here and,
    inside the stack forward, recorded in the store for the layers above; a
    checkpoint recompute reads the forward's record and records nothing.
    detach_sources holds e and the payloads fixed (the autograd check of the
    force, which is the gradient in h_t at fixed sources).
    """
    cfg = self.cfg
    l = int(layer_idx)
    if self._pstore is None:
        raise RuntimeError("particle store: _ps_begin was not called (forward outside
_stack_forward)")
    # E (B, T, K), z (B, T, r)
    E, z, _, k2 = self._ps_match(h, l)
    hb_now = h.float()
    if getattr(cfg, "particle_store_payload", "live") == "detached":
        hb_now = hb_now.detach()
    if getattr(self, "_in_stack_forward", True):
        self._pstore[l] = (E, hb_now)
    entries = [(b, self._pstore[b]) for b in sorted(self._pstore) if b < l] + [(l, (E, hb_now))]
    a = self.pstore_depth[l].float()                                    # (K,)
    K = a.shape[0]
    T = h.shape[1]
    t = torch.arange(T, device=h.device)
    lag = (t[:, None] - t[None, :] - 1)                                 # t - 1 - s
    past = lag >= 0                                                     # s < t
    if self.pstore_logit_lambda is not None:
        log_lam = torch.nn.functional.logsigmoid(self.pstore_logit_lambda.float())
        Lam = torch.where(past, torch.exp(lag.clamp_min(0)[None].float() * log_lam[:, None, None]),
                          torch.zeros((), device=h.device))             # (K, T, T)
        # (T, T, K) [t, s, j]
        Lam = Lam.permute(1, 2, 0)
    else:
        # no decay: every past particle, in full
        Lam = past.float().unsqueeze(-1).expand(T, T, K)
    Pl = self.pstore_P[l].float()                                       # (r, d)
    dd = self.pstore_delta.float() @ Pl.t()                             # (K, r)
    zz = (z * z).sum(-1)                                                # (B, T)
    out = []
    for b, (e_b, h_b) in entries:
        if detach_sources:
            e_b, h_b = e_b.detach(), h_b.detach()
        # (B, T, K, r)  [s, j]
        c = (h_b @ Pl.t()).unsqueeze(-2) + dd
        cc = (c * c).sum(-1)                                            # (B, T, K)
```

```
        zc = torch.einsum("btr,bskr->btsk", z, c)                    # (B, T, T, K)
        d2 = (zz[:, :, None, None] + cc[:, None, :, :] - 2.0 * zc).clamp_min(0.0)
        wu = e_b.unsqueeze(1) * Lam.unsqueeze(0) * torch.exp(-k2 * d2)
        out.append((a * wu, wu, c))
    return z, k2, out
```

The force is (pm.18) assembled from those terms, with the reach statistics recorded in the training forward; the potential is kept alongside for the autograd check:

```
def particle_store_force(self, h: torch.Tensor, layer_idx: int) -> torch.Tensor:
    """-grad_{h_t} U over every particle token t sees, U = -sum w (see _ps_pairs).

    The sources (e, payload) are of tokens s < t, so they are constant in h_t
    and this is the exact gradient in h_t; the loss still reaches them (live).
    -grad U = -2 P_l^T sum_{s,j} w kappa_j^2 (z_t - c_{s,j}).
    """
    with torch.autocast(device_type=h.device.type, enabled=False):
        z, k2, pairs = self._ps_pairs(h, layer_idx)
        g = None
        tot = prev = None
        for b_i, (w, wu, c) in enumerate(pairs):
            W = w * k2                                               # (B, T, T, K)
            s1 = W.sum((2, 3)).unsqueeze(-1) * z                     # (B, T, r)
            s2 = torch.einsum("btsk,bskr->btr", W, c)
            g = (s1 - s2) if g is None else g + (s1 - s2)
            if self.training and getattr(self, "_in_stack_forward", True):
                # reach, independent of the depths
                with torch.no_grad():
                    ws = wu.detach().sum()
                    tot = ws if tot is None else tot + ws
                    if b_i < len(pairs) - 1:
                        prev = ws if prev is None else prev + ws
        f = -2.0 * (g @ self.pstore_P[int(layer_idx)].float())       # (B, T, d)
        if self.training and getattr(self, "_in_stack_forward", True):
            with torch.no_grad():
                # this layer's existences (B, T, K)
                E = self._pstore[int(layer_idx)][0].detach()
                cv = (E.std(dim=(0, 1)) / E.mean(dim=(0, 1)).clamp_min(1e-12)).median()
                share_prev = (prev / tot.clamp_min(1e-30)) if prev is not None else
E.new_tensor(0.0)
                self.pstore_stats[int(layer_idx)] = torch.stack(
                    [E.mean(), cv, share_prev]).to(self.pstore_stats.dtype)
    return f.to(h.dtype)

def particle_store_potential(self, h: torch.Tensor, layer_idx: int) -> torch.Tensor:
    """U(h) = -sum w with every source held fixed: -autograd.grad(U, h) is particle_store_force."""
    with torch.autocast(device_type=h.device.type, enabled=False):
        _, _, pairs = self._ps_pairs(h, layer_idx, detach_sources=True)
        return -sum(w.sum() for w, _, _ in pairs)
```

It enters `_layer_forces` as a plain kick after the Poisson-mode force, so every integrator path — the BAOAB step, PMX, FO-b — receives it in the one force evaluation per layer (Where the Force Enters):

```
# Stage 0'': the particle store's force, a plain kick like the wells.
if self.pstore_mu is not None:
    _f_ps = self.particle_store_force(h_in, layer_idx)
    with torch.no_grad():
        _cons = f_phi if f_theta is None else f_theta + f_phi
        self.pstore_share[layer_idx] = _f_ps.norm() / (_cons.norm() + 1e-12)
    f_phi = f_phi + _f_ps
```

The pass-level lifecycle is two lines in the Fock stack loop and a thin override in the multi-ξ base class, which also sets `_in_stack_forward` as the Fock subclass does:

```
self._ps_begin()        # Stage 0'': an empty particle store for this pass

# Per-layer flow inference mode (cfg.substeps_per_layer, protocol
# SS5.19): k substeps of dt/k per layer; see MultiXiPARFLMConfig.
k_sub = getattr(cfg, "substeps_per_layer", 1)
k_sub = 1 if k_sub is None else int(k_sub)
if k_sub != 1:
    if k_sub < 1:
        raise ValueError(f"substeps_per_layer must be >= 1, got {k_sub}")
    if getattr(self, "pstore_mu", None) is not None:
        raise NotImplementedError("substeps_per_layer > 1 with a particle store: not defined yet")

def _stack_forward(self, h0, x, return_trajectory: bool = False):
    """The base walk, with the particle store emptied at the start of the pass
    and _in_stack_forward set as the Fock subclass sets it (True inside the
    loop only), so that the matcher's state and the store are written by the
    forward and read, not rewritten, by a checkpoint recompute."""
    self._ps_begin()
    self._in_stack_forward = True
    try:
        return super()._stack_forward(h0, x, return_trajectory)
    finally:
        self._in_stack_forward = False
```

The Notebook: Switches, Tag and Guard

Cell 0 gains the switch block, with the tag conditional so that every existing tag is unchanged [2]:

```
# Particle store (Stage 0'', 2026-10-31; Implementing_Semantic_Templates_A_Path.md
# SS7): a write-once, block-causal store of CREATED particles -- the degree of
# freedom a template needs, which neither a slot register (rewritten in place, fixed
# M) nor a Poisson mode (deepens a well at a fixed centre) supplies. PARTICLE_STORE
# = K particle types: at every layer each token bears, per type, a particle with
# existence E_j(h) (the Stage 0' rank-r overlap, its Poisson creation mean) and
# payload h + delta_j, written once; it acts on every later token at this and every
# later layer through a well around the payload (model_parf_multixi.py,
# particle_store_force). Depths start at 0 (force exactly 0 at step 0).
# PARTICLE_STORE_DECAY = False removes the spontaneous death (lambda = 1): particles
# live until consumed and the live count grows with the input; True keeps half-lives
# log-spaced over 1-16 tokens (bounded memory). Explicit kick only. 0 = off
# (bit-identical). Tag 'ps<K>r<rank>' (+ 'nd' without decay).
PARTICLE_STORE       = 0
PARTICLE_STORE_RANK  = 8
PARTICLE_STORE_DECAY = True
PARTICLE_STORE_CLIP  = 0.3

# Stage 0'' particle store: conditional, so every existing tag is unchanged.
if globals().get('PARTICLE_STORE', 0):
    _variant_parts.append(f'ps{PARTICLE_STORE}r{PARTICLE_STORE_RANK}' + ('' if PARTICLE_STORE_DECAY
else 'nd'))
    if abs(PARTICLE_STORE_CLIP - 0.3) > 1e-12:
        _variant_parts.append(f'psclip{PARTICLE_STORE_CLIP:g}'.replace('.', 'p'))
```

Cell 5 maps the three switches onto the configuration (`particle_store`, `particle_store_rank`, `particle_store_decay`); Cell 5b asserts that the tag carries `ps<K>r<rank>` (plus nd without decay) and that the built model has the store, and prints the banner with the half-lives or the no-death notice; Cell 6 puts the `pstore_*` parameters in their own clip group at `PARTICLE_STORE_CLIP`, as the modes have `pm_`:

```
        particle_store=int(globals().get('PARTICLE_STORE', 0)),
        particle_store_rank=int(globals().get('PARTICLE_STORE_RANK', 8)),
        particle_store_decay=bool(globals().get('PARTICLE_STORE_DECAY', True)),

# Stage 0'': the particle store's parameters (pstore_*) in their own group;
# 'pstore_' is a substring of no other parameter name.
```

```
if globals().get('PARTICLE_STORE', 0):
    GRAD_CLIP_OVERRIDES['pstore_'] = PARTICLE_STORE_CLIP
```

Verification of the Store

`verify_particle_store.py` [14] follows `verify_pm_match.py`. Bit-identity with the switch off was checked by that script's fingerprint on a checkout of HEAD and on the working copy: the four configurations (F3.1, PM1-K64, PMX-K16, FO-b-PMX-K16) agree on the loss, the logits checksum, every parameter and every gradient. The checks with the store on (K = 4, rank 8, F3.1 base with SR2, explicit kick; CPU, at initialisation, real validation tokens), all passed:

- the tag carries ps4r8; the new parameters are exactly the six pstore_* tensors, and the clip group pstore_ catches them and nothing else;
- with the depths at zero the eval and train logits are bit-identical to the model without the store, and the gradient reaches pstore_depth;
- the first training forward initialises every layer's matcher from its states; the existences are finite and in (0, 1);
- with the depths and offsets set non-zero, the force equals $-\nabla U$ by autograd at fixed sources to about 1e-7, at layer 0 (own particles) and at layer 1 (layer-0 and own particles);
- causality: tokens replaced at t ≥ 128 leave the logits at t < 128 unchanged;
- layer checkpointing on and off give the same loss and the same gradients, bit for bit, from the same state — for the store and for the Stage 0' matcher (next subsection);
- every pstore_ parameter has a finite, non-zero gradient.

The readings for criterion 2 of the stage — number dynamics that vary with content — at initialisation:

arm	layer	mean existence	CV across tokens	reach from earlier layers	live particles at t = 32 / 128 / 255
decay 1–16	0	0.292	0.56	—	11.0 / 15.0 / 14.2
decay 1–16	1	0.286	0.58	53%	13.7 / 16.5 / 16.1
no decay	0	0.292	0.56	—	34.7 / 148.3 / 298.5
no decay	1	0.286	0.58	58%	35.2 / 145.7 / 292.2

Two of the four types are broad (a reach of 64–70% of tokens) and two narrow (2–4%), the k-means geometry of Appendix A6 at K = 4. The coefficient of variation of the existence across tokens, 0.56–0.58, is the Stage 0' selectivity carried over. Layer-0 particles carry 53–58% of the reach at layer 1: cross-layer particles are seen, which the layer-0 / layer-1 cloud mismatch of Why the Trained Modes Cannot Match had put in doubt. The no-decay arm passes the structural half of criterion 2 (the count grows with t); whether training keeps the existences content-dependent is the short-training question, as it was for the matcher.

A Pre-Existing Bug in the Stage 0′ Matcher Under Checkpointing

Checkpoint identity failed on the first run of the checks, and the cause was not the store. `_lowrank_match` updates the running median D_ℓ^2 of (pm.15) in every training forward — including the recompute that layer checkpointing runs in the backward. The recompute then saw a different width than the forward had used, computed a different force, and moved the median a second time per step: on PM1-K16 with `POISSON_MATCH_RANK = 8` the gradients differed by 6.6×10^{-5} between checkpointing on and off, and D^2 after one step was 220.9 against 293.9. The production default is `LAYER_CHECKPOINT = True`. The state now moves only in the training forward inside the stack loop:

```
# The initialisation and the running median are state: they run in the
# training forward inside the stack loop only. A layer-checkpoint recompute
# in the backward (_in_stack_forward False) must see the D2 the forward
```

```
# used, or it recomputes a different force than the forward applied and
# moves D2 twice per step (2026-10-31: 6.6e-5 in the gradients, D2 220.9
# vs 293.9 after one step on PM1-K16 pmlr8; debug/verify_particle_store.py).
if self.training and getattr(self, "_in_stack_forward", True):
```

Both arms are bit-identical with checkpointing on and off after the change, and the fingerprint of the four configurations without checkpointing is unchanged. The planned 2B template pair runs with `LAYER_CHECKPOINT = False` (The Planned Template Pair) and was never exposed; any `pmlr` run under the default carried the inconsistency.

What the First Build Does Not Yet Do

- **Train.** The short CPU run of pm_match_short_train.py, in both decay arms, is the next measurement: does the existence stay content-dependent, and does the model use the particles (content swap)?
- **Consume.** Nothing has a reactant yet. The collapse-depth criterion of the stage — the Dyck-1 depth D* of the bounds note moving with the training length — belongs to the first consuming template, because without consumption a closing bracket cannot decrement the live count.
- **Move.** The payload is frozen at birth. The alternative, integrating created particles under the flow as Fock v2 did when it concatenated the registers into the Verlet state, is open; write-once is the simpler first choice, not the only one.
- **Learn the bank.** The Tier 1 training algorithm — which templates exist, their rates and thresholds — is the design step that follows; its inputs are the store and the matcher, both now in place.

A store arm — PMX-K16's base with `PARTICLE_STORE = 4` and no decay, tag …ps4r8nd… — is the natural third member of the 2B template pair once its short run is in.

The 2B-Token Run

Why 2B, and What It Must Decide

The calls of the book rest on 0.53B-token runs. Using a memory of content is a learned behaviour that may appear late, so a negative at 0.53B would not separate the design from the budget; the isotropic wells already gave one such result (broad and content-blind at 0.5B). A 2B run with snapshots gives an emergence curve instead of an endpoint. It decides three things: whether the potential still falls along every trained step (the basis of a native hallucination detector), whether the trained map moves toward or away from its continuous law as training lengthens, and whether the wells start to remember content.

The Schedule and the Snapshots

Each 2B arm uses its 0.5B Cell 0 plus `LADDER_STEPS = 130_000`: 2.13B tokens, about 1.07 passes over the 2.0B-token pool. The WSD fractions are unchanged (`WSD_WARMUP_FRAC = 0.05`, `WSD_STABLE_FRAC = 0.60`), so warm-up ends at step 6,500, the stable phase at 84,500 and the decay at 130,000, with peak learning rate 1.2×10^{-3} and floor 6×10^{-5}.

Snapshot Code (Cell 6)

`SNAPSHOT_STEPS` lists the steps at which a weights-only checkpoint is written (no optimizer state, about 0.3 GB each), named `<prefix>_step<N>_snap.pt`, kept for good and never used to resume. The 2B grid is

```
[6_500, 16_500, 32_500, 48_500, 65_000, 84_500, 97_500, 113_500, 130_000]
```

the end of warm-up; four points in the stable phase, of which 32,500 is the 0.5B arms' token count; the start of the decay; two in the decay; and the end. With the default `[]` the save branch is never entered:

```
# training-trend snapshot (2026-10-22): weights only, kept, never used to resume
if (step + 1) in set(globals().get('SNAPSHOT_STEPS', []) or []):
    if (step + 1) % EVAL_INTERVAL != 0:
        val_loss = evaluate()
```

```
        val_ppl = math.exp(val_loss)
    _snap = CKPT_DIR / f'{CKPT_PREFIX}_step{step + 1}_snap.pt'
    try:
        torch.save({'model_state_dict': model.state_dict(), 'model_cfg': asdict(model_cfg),
                    'step': step + 1, 'val_loss': val_loss, 'val_ppl': math.exp(val_loss),
                    'snapshot': True, 'seed': SEED, 'corpus': CORPUS}, _snap)
        print(f'  Snapshot saved: {_snap.name}  (PPL={math.exp(val_loss):.2f}, weights only)')
    except OSError as _e:
        print(f'[WARN] Snapshot NOT saved at step {step + 1}: {_e}')
```

Reading a Snapshot: ckpt= and the Harness

Every CPU tool loads a model through one function, `verify_pm_switch.build`, which executes the notebook's own Cells 0–5b with the arm's Cell 0 replacements and loads a checkpoint from the run folder. It takes a file name, so any snapshot can be read:

```
def build(folder, repl, ckpt=None):
    """ckpt: a checkpoint file name in <folder>/checkpoints (default <prefix>_best.pt), e.g. a
    <prefix>_step<N>_snap.pt training-trend snapshot (2026-10-22)."""
    c0 = G.cells['Cell 0:']
    for old, new in repl:
        assert c0.count(old) == 1, old
        c0 = c0.replace(old, new)
    os.chdir(G.REPO / 'notebooks/conservative_arch/scaleup')
    g = {'__name__': '__main__'}
    out = io.StringIO()
    with contextlib.redirect_stdout(out):
        exec(compile(G.strip(c0), 'Cell0', 'exec'), g)
        exec(compile(G.strip(G.cells['Cell 1:']), 'Cell1', 'exec'), g)
        g['CKPT_DIR'] = Path.home() / 'Downloads' / folder / 'checkpoints'
        g['RESULTS_DIR'] = G.OUT / 'results'; g['RESULTS_DIR'].mkdir(parents=True, exist_ok=True)
        g['DATA_DIR'] = G.OUT / 'data'; g['GDRIVE_ROOT'] = G.OUT
        exec(compile(G.strip(G.cells['Cell 1b:']), 'Cell1b', 'exec'), g)
        g['DATA_DIR'] = G.OUT / 'data'
        exec(compile(G.strip(G.cells['Cell 2:']), 'Cell2', 'exec'), g)
        exec('from data_module import get_batch', g)
        g['val_ids'] = np.load(G.VAL); g['train_ids'] = g['val_ids']
        exec(compile(G.strip(G.cells['Cell 4:']), 'Cell4', 'exec'), g)
        exec(compile(G.strip(G.cells['Cell 5:']), 'Cell5', 'exec'), g)
        g['PROBE_MAX_STEPS'] = g.get('PROBE_MAX_STEPS')
        exec(compile(G.strip(V.c5b), 'Cell5b', 'exec'), g)
    model = g['model']
    pfx = folder[len('semsimula_'):].replace('fock_cfc_baoab_owt', 'fock_cfc_owt')
    ck = torch.load(g['CKPT_DIR'] / (ckpt or f'{pfx}_best.pt'), map_location='cpu',
weights_only=False)
```

Per-Snapshot Diagnostics

At every snapshot of every arm, on CPU:

id	measurement
D-a	perplexity on Cell 6b-7's 12 fixed batches
D-b	the per-layer flow: penalties at $N = 4$–16, convergence, standard refinement
D-c	the descent readout: direction and distance against the continuous law, the potential along the trained step, a force-based basin
D-d	$\omega\Delta t$ of each token's stiffest mode, per layer

id	measurement
D-e	the wells: depths, half-lives, force shares, N_{eff}; for the template pair also selectivity, the position share and the content-swap cost
D-f	Gates 1–3 at 32,500, 84,500 and 130,000

The Planned Template Pair

The trained test of Stage 0' is a pair at 2B tokens, identical except for the matcher, both on PMX-K16's base with explicit wells and the 1–16 range:

arm	tag	parameters
template	…pm16_pmhl1t16_pmlr8_sr2_st130000…	76,590,622
isotropic control	…pm16_pmhl1t16_sr2_st130000…	76,578,318

Cell 0 for both: PMX-K16's list (`REVERSE_CHANNEL = False`; `VPHI_GRAD_PATH`, `XI_GRAD_PATH = 'live'`; `POISSON_MODES = 16`; `LOWRANK_DAMPED_FLOW = True`; `VPHI_ANALYTIC_FORCE = True`; `LAYER_CHECKPOINT = False`) with `POISSON_WELLS_EXACT` left `False`, plus `POISSON_HALFLIFE_MIN = 1.0`, `POISSON_HALFLIFE_MAX = 16.0`, `LADDER_STEPS = 130_000` and the snapshot list; the template arm adds `POISSON_MATCH_RANK = 8`. The configurations are built and checked by:

```
if MODE == 'planned':
    SNAP = "SNAPSHOT_STEPS = [6_500, 16_500, 32_500, 48_500, 65_000, 84_500, 97_500, 113_500,
130_000]"
    RUN = (("LADDER_STEPS     = 32_500", "LADDER_STEPS     = 130_000"), ("SNAPSHOT_STEPS = []",
SNAP),
           ("POISSON_HALFLIFE_MIN = 4.0", "POISSON_HALFLIFE_MIN = 1.0"),
           ("POISSON_HALFLIFE_MAX = 128.0", "POISSON_HALFLIFE_MAX = 16.0"))
    for name, extra in (('template arm', (("POISSON_MATCH_RANK   = 0", "POISSON_MATCH_RANK   =
8"),)), ('isotropic control', ())):
        model, g = build_init(F31 + K(16) + SR2 + RUN + extra)                  # runs the
notebook's Cell 5b
        model.to('cpu'); model.train(); model.cfg.use_layer_checkpoint = False
        hl = (-np.log(2) / torch.nn.functional.logsigmoid(model.pm_logit_lambda.detach()).numpy())
        xb, yb = tokens(g)
        with torch.enable_grad():
            _, loss = model(xb, targets=yb)
        loss.backward()
        fin = bool(torch.isfinite(loss)) and all(torch.isfinite(p.grad).all() for p in
model.parameters() if p.grad is not None)
        tag = g['_variant_tag']
        print(f'  {name:17s} tag ...{tag[tag.find("_pm16"):tag.find("_L2probe")]}_L2probe...  steps
{g["LADDER_STEPS"]:,}  '
              f'snapshots {len(g["SNAPSHOT_STEPS"])}  half-lives {hl.min():.2f}-{hl.max():.2f}
tokens  '
              f'match rank {model.cfg.poisson_match_rank}  {sum(p.numel() for p in
model.parameters()):,} params  '
              f'loss {float(loss):.4f}  finite {fin}  (Cell 5b passed)', flush=True)
        del model
    sys.exit(0)
```

Pre-Registered Calls

The 2B calls were registered before any 2B run (protocol §5.13, TL) [11]:

id	prediction	called
TL1	the potential falls along the trained step for at least 99% of tokens at	80%

id	prediction	called
	both layers, at every snapshot from 16,500 on	
TL2	the per-layer flow penalty at $N = 16$ is larger at 84,500 than at 32,500 in at least three of four arms	60%
TL3	first order departs from its continuous law more than second order, in both pairs	55%
TL4	layer 0 stays a descent: the first-order arms' median direction cosine at least 0.95	70%
TL5	the second-order arm gains more in the decay phase, in both pairs	65%
TL6	SR2-2B is more than 4.9% better than FO-b-2B	50%
TL7	N_{eff} rises from 32,500 to 84,500 in both $K = 64$ arms	55%
TL8	N_{eff} at 130,000 exceeds the 0.5B value in both $K = 64$ arms	60%
TL9	the first-order arm's N_{eff} exceeds its twin's at both layers	55%
TL10	the content swap costs more than 0.5% at 130,000 in at least one $K = 64$ arm	35%

The template pair's own calls are to be registered when it is scheduled.

Paired Evaluation: the Shared Random Stream

One generator, seeded by `SEED`, serves both the training batches and the evaluation batches (Cell 6: `rng = np.random.default_rng(SEED)`; `evaluate()` draws from the same `rng`). Every run trained from step 0 at the same seed therefore sees the same training data in the same order and the same evaluation batches at each step, and comparisons between such runs are paired. The step-to-step changes of the evaluations correlate at 0.92–0.99 across arms, and a paired difference between two arms is resolved to 0.1–0.3% where the unpaired settled values carry about ±1.2% of evaluation noise (Appendix A8). A resumed run (PMX at $K = 64$, resumed at step 8,500) draws different batches after the resume. The template pair is to be trained from step 0, so that its two arms are paired.

References

[1] notebooks/conservative_arch/parf/model_parf_multixi.py

[2] notebooks/conservative_arch/scaleup/colab_fock_cfc_baoab_lowrank_depth_ladder_openwebtext_d384.ipynb

[3] Semantic Simulation: Prescriptive Lagrangian Framework, D. Gueorguiev, paper v6, §9 (Poisson-mode registers) and §26.9 (memory substitution), 2026

[4] Deep Dive in the SPLM model and code, D. Gueorguiev, 2026 (companion_notes/deep_dives)

[5] The Single-Particle Hilbert Space in the Semantic Simulation Framework, companion note

[6] notebooks/conservative_arch/parf/cfc_baoab.py

[7] notebooks/conservative_arch/scaleup/debug/ (pm_mode_usage.py, pm_content_swap.py, pm_static_landscape.py, mechanism_ablations.py, vtheta_omega_dt.py, run_diag_cell_cpu.py)

[8] Implementing Semantic Templates: A Path, companion note; and Semantic Templates as Doi–Peliti Reactions

[9] debug/narrow_wells_design.py and its output narrow_wells_design_output.txt

[10] debug/verify_pm_match.py, debug/pm_match_short_train.py, debug/pm_halflife_range.py

[11] Depth Ladder and Matched Baseline Protocol, §5.13 (MS, TL, the K = 16 pair and the wells' results)

[12] A Community of Semantic Templates as the Source of Native Chain-of-Thought, companion note

[13] Doi–Peliti Dynamics of Semantic Particles and Registers, companion note

[14] debug/verify_particle_store.py and its output verify_particle_store_output.txt

[15] Expressivity Bounds for the v0 Simulator, companion note (Lemma 2, Lemma 4, §8.2)

[16] MCS Reduction for the v3 Composite, companion note

Appendix

A1. The Poisson Thinning Argument

Let N_s be the number of particles that token s creates in mode v, $N_s \sim \mathrm{Poisson}(E_v(s))$, and let each survive every later token independently with probability λ_v. The survivors from token s at token $t > s$ are a binomial thinning of N_s with retention $q = \lambda_v^{t-1-s}$. The probability generating function of a Poisson variable of mean E is $G(z) = e^{E(z-1)}$; thinning replaces z by $1 - q + qz$, so

$$G_{\text{thinned}}(z) = e^{E\,((1-q+qz)-1)} = e^{qE\,(z-1)} \qquad \text{(a1.1)}$$

which is Poisson with mean qE. Survivors from different tokens are independent, and a sum of independent Poisson variables is Poisson with the summed mean, $\sum_{s<t} \lambda_v^{t-1-s}\, E_v(s) = \bar{n}_v(t)$. The modes are independent by construction, so the joint law is a product of Poisson laws: the coherent state $e^{\sum_v \bar{n}_v(t)(a_v^\dagger - 1)}|0\rangle$ [13].

Two consequences used in the main text. Since V_{pm} is linear in the particle numbers, $\mathbb{E}[V_{\text{pm}}] = V_{\text{pm}}(\bar{n})$ and the same holds for the force: the mean is exact, not mean-field. And with a constant creation rate E the occupation approaches the stationary value

$$\bar{n}_v^* = \frac{E}{1-\lambda_v} \qquad \text{(a1.2)}$$

the Poisson mean c/d of spontaneous creation (rate $c = E$ against destruction $d = 1 - \lambda_v$ per token), which is the prior of templates with trivial matchers (*What a Template Needs*).

A2. The Exact Damped Flow of an Indefinite Spring

Along one direction with stiffness k (either sign), mass $\mathfrak{m}$ and damping γ, the A substep integrates $\ddot{x} + \gamma\dot{x} + \omega_0^2 x = a$ with $\omega_0^2 = k/\mathfrak{m}$ and constant forcing $a = f/\mathfrak{m}$, from $x(0) = 0$, $\dot{x}(0) = v$. With $\omega_d^2 = \omega_0^2 - \gamma^2/4$ (either sign), $\rho = \sqrt{|\omega_d^2|}$, $C = \cos\rho t$ and $S = \sin(\rho t)/\rho$ when $\omega_d^2 \geq 0$, and $C = \cosh\rho t$, $S = \sinh(\rho t)/\rho$ otherwise,

$$x(t) = E_{12}\, v + Q\, a, \qquad \dot{x}(t) = E_{22}\, v + P\, a \qquad \text{(a2.1)}$$

$$E_{12} = e^{-\gamma t/2} S, \qquad E_{22} = e^{-\gamma t/2}(C - \gamma/2\, S), \qquad P = E_{12}, \qquad Q = \frac{1 - e^{-\gamma t/2}(C + \gamma/2S)}{\omega_0^2} \qquad \text{(a2.2)}$$

`damped_mode_coefficients` evaluates these in float64 for every regime (underdamped, overdamped, critical, $\omega_0 = 0$), summing Q from its Taylor recurrence where $\omega_0^2 t^2$ is small, because the division cancels catastrophically there. A negative stiffness, the top of a repulsive well, gives the hyperbolic branch: the direction grows, exactly.

For the spring $K = \alpha I + U\,\mathrm{diag}(\kappa)\,U^\top$ with orthonormal columns U, the space splits into $\mathrm{span}(U)$, where direction q has stiffness $\kappa_q + \alpha$, and its complement, where every direction has stiffness α. `lowrank_iso_damped_substep` projects the velocity and force onto each part, applies (a2.1) with the corresponding stiffness, and adds the results. The damping is inside the flow on all of $\mathbb{R}^d$, so no friction step may follow.

A3. The Overdamped Relaxation

The first-order flow $\dot{x} = \mu_{\mathrm{FO}}(f - kx)$ from $x(0) = 0$ with constant f has the solution

$$x(t) = \mu_{\mathrm{FO}}\, t\, \varphi(\mu_{\mathrm{FO}} k t)\, f, \qquad \varphi(y) = \frac{1 - e^{-y}}{y} \qquad \text{(a3.1)}$$

with $\varphi(0) = 1$ (a free step) and $\varphi(y) \approx 1/y$ for large y (full relaxation to the spring's equilibrium f/k). `_relax_coeff` evaluates φ smoothly through $y = 0$ and for either sign; a negative stiffness grows as $e^{\mu_{\mathrm{FO}}|k|t}$. The relaxation exponent $\mu_{\mathrm{FO}} k \Delta t$ is FO-b's own stiffness measure: above about 3 a mode relaxes fully within the step.

A4. Concentration of Distances and Low-Rank Matching

Let the token states near a centre behave, for distances, like a cloud with d_{eff} independent directions of comparable variance σ^2. Then $\|h - \mu\|^2$ is a sum of d_{eff} comparable terms, with mean $\approx d_{\mathrm{eff}}\sigma^2$ (plus the squared offset of the centre) and relative standard deviation of order $\sqrt{2/d_{\mathrm{eff}}}$. A Gaussian kernel $E = \exp(-c\,\|h - \mu\|^2 / D^2)$, with D^2 the median squared distance, then maps the tokens to

$$\frac{E_{\mathrm{near}}}{E_{\mathrm{far}}} \approx \exp(c \cdot \mathrm{spread}) \qquad \text{(a4.1)}$$

with *spread* the relative width of the distance distribution. For a full-rank cloud the spread is small (0.20–0.24 measured), so the kernel is either almost constant over the cloud (c small) or almost zero except at a few near neighbours (c large): it cannot grade. In a subspace of r directions the spread grows roughly as $1/\sqrt{r}$ (measured 0.58–0.60 at $r = 16$, 0.77–1.00 at $r = 8$), and a width exists at which each mode reaches a graded fraction of the tokens. The principal directions are the right subspace at initialisation because they carry the cloud's largest variations; P_ℓ is learned afterwards.

A5. How an Occupation Averages Content Away

Write a mode's overlap as $E_v(s) = \bar{E}_v + \delta_v(s)$, with δ_v the token-level (content) part, of variance σ_v^2 and short correlation across positions. The occupation (pm.3) is then

$$\bar{n}_v(t) = \bar{E}_v\, \frac{1 - \lambda_v^{t-1}}{1 - \lambda_v} + \sum_{k \geq 0} \lambda_v^k\, \delta_v(t - 1 - k) \qquad \text{(a5.1)}$$

The first term depends on position only. The second carries content, with variance $\sigma_v^2(1 - \lambda_v^{2(t-1)})/(1 - \lambda_v^2)$. Far from the start, the ratio of the content part's standard deviation to the occupation's mean is

$$\frac{\mathrm{std(content)}}{\mathrm{mean}} \approx \frac{\sigma_v}{\bar{E}_v}\sqrt{\frac{1 - \lambda_v}{1 + \lambda_v}} \qquad \text{(a5.2)}$$

which vanishes as $\lambda_v \to 1$. For a half-life of 128 tokens the square root is about 0.052; for 2 tokens about 0.41. Even with selective overlaps ($\sigma_v/\bar{E}_v \approx 1$), long half-lives average content into a small ripple on a positional curve, and the early positions, where the positional term still rises, add positional variance that dominates. That is the trend of the measured position shares (*The Half-Life Range*), and it is why the template pair starts from 1–16 tokens.

A6. The k-Means Initialisation and the Clustered Initial Cloud

At initialisation the layer-0 states are the token embeddings, drawn independently, and a token that occurs often becomes a dense point repeated many times. The principal directions of the cloud point at the most frequent of these, so the projected cloud is a dense core (about 75% of the tokens, the rest of the vocabulary) plus small, tight clusters (2–4% each). k-means++ seeding samples centres with probability proportional to the squared distance from the centres already chosen, which reaches the clusters; ten Lloyd passes then move each centre to the mean of its cluster. At $K = 16$ about five centres sit in the core and eleven in the clusters, so the median mode reaches a small cluster while every token reaches some mode (coverage 100%). This geometry belongs to the random embeddings and changes in training; the short training run shows the selectivity surviving the change.

A7. The Fixed Landscape: What the Static Test Shows and What It Does Not

The static test replaces $\bar{n}_v(t)$ by $\bar{n}_v^{\text{late}}$, its mean over positions 256–511, at every position. Its cost falls with position (19% in the first 16 tokens, nothing from 128 on), while the cost of removing the wells grows with position. Read together: past the first few half-lives the occupations are constant, the wells reduce to the fixed potential $-\sum_v \bar{n}_v^{\text{late}}\, a_{\ell,v} e^{-\kappa_v^2 \|h-\mu_v\|^2}$, and the model depends on that potential heavily. The growing removal cost reflects the model's reliance on a term it was trained with (reliance, not value: trained-without gains are much smaller than removal costs throughout this series), not wells that keep deepening. What the test does not show is whether the fixed potential needs the Poisson machinery at all; a first-order model with a fixed bank of K Gaussian wells per layer would answer that (proposed in the protocol).

A8. Paired Evaluation and the Seed

Let $\ell_A(n)$ and $\ell_B(n)$ be two runs' evaluation losses at step n, each the true loss plus an evaluation-sampling error $e(n)$ that depends on the batches drawn. If the runs draw the same batches, $e_A(n) \approx e_B(n)$ and the paired difference $\ell_A(n) - \ell_B(n)$ cancels most of it; measured, the paired spread over the last eight evaluations is 0.08–0.23% against about 2% per evaluation unpaired. A second seed changes the initialisation, the data order and the evaluation batches together, so it loses the pairing; measuring the seed spread (MS1) cleanly needs a separate evaluation generator, seeded 0 and advanced through the same draws as seed 0's training batches, so that a seed-1 run replays seed 0's evaluation batches. That switch is planned, not yet built.